



N O S
Cités
ANALYSE URBAINE

Analyse des locations courte durée à Paris & Lyon

Restauration & Analyse MongoDB | Été 2024 | Association NosCités

[Morgan Le Gall | Association NosCités | Mars 2026]

Sommaire

Introduction : Contexte & Importation

Contexte du projet

Importation des données

Partie 1 — Structure des données, NoSQL & Premières requêtes

1-1 Structure d'un document

1-2 Pourquoi NoSQL ?

1-3 Premières requêtes de validation

Partie 2 — Requêtes & Résultats d'analyse

2-1 Requêtes simples via mongosh

2-2 Requêtes complexes via Python + Polars

2-3 Mise à disposition BI

Partie 3 — Architecture haute disponibilité

3-1 Schéma de la base de données MongoDB

3-2 Fusion des collections Paris + Lyon

3-3 Réplication — ReplicaSet noscitesRS

3-4 Distribution — Sharding par zone géographique

4 — Synthèse

Cartographie des 10 ports (27017-27029)

Tableau des résultats clés Paris vs Lyon

Scripts de démarrage et d'arrêt

5 — Utilisation pratique

Conclusion

Contexte du projet et importation des données

Contexte du projet

Contexte du projet



NosCités collecte et analyse des **données** sur les **locations courte durée** à **Paris et Lyon** afin d'étudier l'impact du tourisme sur les marchés locatifs.

Contexte du projet



NosCités collecte et analyse des **données** sur les **locations courte durée** à **Paris et Lyon** afin d'étudier l'impact du tourisme sur les marchés locatifs.



Crash de la base de données Paris.

Contexte du projet



NosCités collecte et analyse des **données** sur les **locations courte durée** à **Paris et Lyon** afin d'étudier l'impact du tourisme sur les marchés locatifs.



Crash de la base de données Paris.



Sauvegarde retrouvée et importée en urgence pour l'équipe parisienne.

Objectif

- Restaurer et importer les données depuis les fichiers CSV sources (Paris et Lyon)

Objectif

- Restaurer et importer les données depuis les fichiers CSV sources (Paris et Lyon)
- Valider l'intégrité des données via des requêtes d'exploration et d'analyse

Objectif

- Restaurer et importer les données depuis les fichiers CSV sources (Paris et Lyon)
- Valider l'intégrité des données via des requêtes d'exploration et d'analyse
- Mettre en place une architecture robuste : réplication (ReplicaSet) et distribution (Sharding) pour prévenir tout incident futur

Importation des données

📌 **Données sources : listings_Paris.csv** (185 Mo, 95 885 lignes) et **listings_Lyon.csv** (19 Mo, 9 973 lignes).

Chaque enregistrement représente une annonce Airbnb avec 75 champs : localisation, prix, disponibilité, avis, type de logement, etc.

```
moylig@DESKTOP-CUIQHFG:/mnt/c/Users/moymo/OC/P7$ mongoimport \
--db noscites \
--collection listings \
--type csv \
--headerline \
--file "listings_Paris.csv"
2026-03-16T12:15:07.503+0100    connected to: mongodb://localhost/
2026-03-16T12:15:10.503+0100    [#####.....] noscites.listings    42.0MB/185MB (22.7%)
2026-03-16T12:15:13.503+0100    [#####.....] noscites.listings    85.1MB/185MB (46.1%)
2026-03-16T12:15:16.503+0100    [#####.....] noscites.listings    129MB/185MB (69.9%)
2026-03-16T12:15:19.503+0100    [#####.....] noscites.listings    171MB/185MB (92.7%)
2026-03-16T12:15:20.485+0100    [#####.....] noscites.listings    185MB/185MB (100.0%)
2026-03-16T12:15:20.486+0100    95885 document(s) imported successfully. 0 document(s) failed to import.
```

01

Structure des données, NoSQL & Premières requêtes

1-1 Structure d'un document

75 champs : id, host_id, host_name, host_is_superhost (bool), neighbourhood_cleansed (quartier), room_type, property_type, price, availability_30/60/90/365, number_of_reviews, instant_bookable, reviews_per_month.

Champs semi-structurés : amenities (liste JSON), description, host_verifications (tableau).

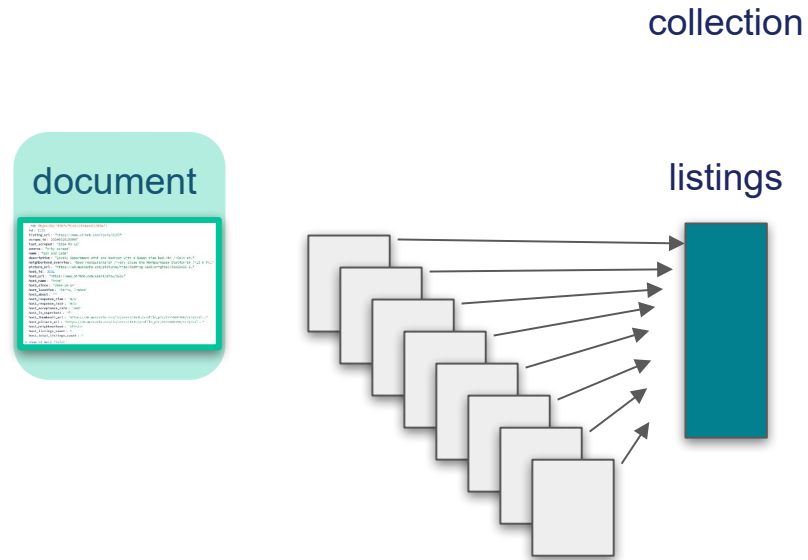
```
_id: ObjectId('69b7e781db116f4abd116f04')
id: 3109
listing_url: "https://www.airbnb.com/rooms/3109"
scrape_id: 20240610195007
last_scraped: "2024-06-12"
source: "city scrape"
name: "zen and calm"
description: "Lovely Apartment with one bedroom with a Queen size bed.<br />Calm et..."
neighborhood_overview: "Good restaurants<br />very close the Montparnasse Station<br />15 m fr..."
picture_url: "https://a0.muscache.com/pictures/miso/Hosting-3109/original/50c69430-6..."
host_id: 3631
host_url: "https://www.airbnb.com/users/show/3631"
host_name: "Anne"
host_since: "2008-10-14"
host_location: "Paris, France"
host_about: ""
host_response_time: "N/A"
host_response_rate: "N/A"
host_acceptance_rate: "80%"
host_is_superhost: "f"
host_thumbnail_url: "https://a0.muscache.com/im/users/3631/profile_pic/1375800198/original..."
host_picture_url: "https://a0.muscache.com/im/users/3631/profile_pic/1375800198/original..."
host_neighbourhood: "Alésia"
host_listings_count: 1
host_total_listings_count: 2
▼ Show 51 more fields
```

1-1 Structure d'un document

document

```
1  # -*- coding: utf-8 -*-
2  """
3  Module de gestion des documents.
4  """
5  from datetime import datetime
6  from random import randint
7  from string import ascii_letters
8  from sys import argv
9  from time import sleep
10
11  # Paramètres
12  nb_documents = 10
13  nb_caractères = 10
14
15  # Fonction pour générer un document
16  def generer_document():
17     """Génère un document aléatoire de nb_caractères de long et le retourne sous forme de chaîne de caractères"""
18     document = ""
19     for i in range(nb_caractères):
20         document += chr(randint(0, 255))
21     return document
22
23  # Fonction pour sauvegarder un document
24  def sauvegarder_document(document):
25     """Sauvegarde un document dans un fichier"""
26     fichier = "document_{}.txt".format(randint(0, 1000000))
27     with open(fichier, "w") as f:
28         f.write(document)
29
30  # Génération et sauvegarde de documents
31  for i in range(nb_documents):
32     document = generer_document()
33     sauvegarder_document(document)
34     sleep(0.5)
35
36  # Affichage des documents sauvegardés
37  for i in range(nb_documents):
38     fichier = "document_{}.txt".format(randint(0, 1000000))
39     with open(fichier, "r") as f:
40         document = f.read()
41         print(document)
42         sleep(0.5)
43
44  # Fin du programme
45  print("Fin du programme")
```

1-1 Structure d'un document



1-1 Structure d'un document



1-2 Pourquoi NoSQL ?

- Le modèle document correspond à la nature réelle de la donnée — une annonce n'est pas une ligne, c'est un document composite et évolutif.
- Le NoSQL n'est pas un choix par défaut, c'est le choix aligné avec la structure, le volume et la trajectoire de croissance de la plateforme.

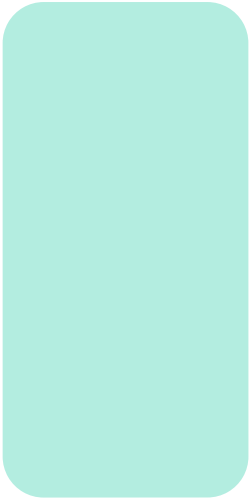


Schéma intrinsèquement variable
Imbrication native sans jointure
Évolutivité horizontale sans migration
Volume et distribution : le sharding comme réponse native
Modèle de requête aligné sur le modèle applicatif

1-3 Premières requêtes

Résultat : 95 885 documents importés | 75 champs | Base : noscites | Collection : listings

```
moylig@DESKTOP-CUIQHFG:/mnt/c/Users/moymo/OC/P7$ mongosh --port 27017
Current Mongosh Log ID: 69b7ebc70b333d9fea8563b0
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.7.0
Using MongoDB:      8.0.19
Using Mongosh:       2.7.0
```

```
test> use noscites
switched to db noscites
noscites> db.listings.countDocuments()
95885
```

```
noscites> db.listings.countDocuments({ has_availability: "t" })
90173
```

1-3 Premières requêtes

```
noscites> db.listings.findOne()
```

```
[direct: mongos] noscites> db.listings.findOne()
{
  _id: ObjectId('69b92bc70ba00b124d958a71'),
  id: 56766,
  listing_url: 'https://www.airbnb.com/rooms/56766',
  scrape_id: Long('20240618214414'),
  last_scraped: '2024-06-19',
  source: 'previous scrape',
  name: 'Duplex avec vue, centre ville',
  description: "Beautiful duplex terrace on Lyon's roofs ! Comfort, design and charming view in the historical Lyon's area. Main museums, theaters, opera, shops and restaurants are located from 2 to 10 min. by walk. Parking and main public transport are at 2 min",
  neighborhood_overview: "the general atmosphere, the magnificent architecture of the Renaissance buildings, the Quai Saint-Atoine market, one of the most beautiful in Lyon, the magnificent St. John's Cathedral and the Gallo-Roman theater",
  picture_url: 'https://a0.muscache.com/pictures/26675020/4940f88e_original.jpg',
  host_id: 269557,
  host_url: 'https://www.airbnb.com/users/show/269557',
  host_name: 'Isabelle',
  host_since: '2010-10-24',
  host_location: 'Lyon, France',
  host_about: "I'm 47 years old. I enjoy books and arts. I'm from Lyon and I would be glad to welcome you.",
  host_response_time: 'within an hour',
  host_response_rate: '100%',
  host_acceptance_rate: '50%',
  host_is_superhost: 'f',
  host_thumbnail_url: 'https://a0.muscache.com/im/users/269557/profile_pic/1322994596/original.jpg?aki_policy=profile_small',
  host_picture_url: 'https://a0.muscache.com/im/users/269557/profile_pic/1322994596/original.jpg?aki_policy=profile_x_medium',
  host_neighbourhood: '5th Arrondissement',
  host_listings_count: 1,
  host_total_listings_count: 3,
  host_verifications: "['email', 'phone', 'work_email']",
  host_has_profile_pic: 't',
  host_identity_verified: 't',
  neighbourhood: 'Neighborhood highlights',
  neighbourhood_cleansed: '5e Arrondissement',
  neighbourhood_group_cleansed: '',
  latitude: 45.76312,
  longitude: 4.82813,
  property_type: 'Entire rental unit',
  room_type: 'Entire home/apt',
  accommodates: 6,
  bathrooms: '',
  bathrooms_text: '2.5 baths',
  bedrooms: 3,
```

02

Requêtes & Résultats d'analyse

2-1 Requêtes simples

mongosh

```
mongosh --port 27017
```

```
use noscites
```

Q1 — Annonces par type de location

```
noscites> db.listings.aggregate([
|   { $group: { _id: "$room_type", count: { $sum: 1 } } },
|   { $sort: { count: -1 } }
| ])
[
|   { _id: 'Entire home/apt', count: 85733 },
|   { _id: 'Private room', count: 8975 },
|   { _id: 'Hotel room', count: 776 },
|   { _id: 'Shared room', count: 401 }
| ]
```

2-1 Requêtes simples

mongosh

Q2 — Top 5 annonces avec le plus d'évaluations

```
noscites> db.listings.find(
|   {},
|   { id: 1, name: 1, number_of_reviews: 1, _id: 0 }
| ).sort({ number_of_reviews: -1 }).limit(5)
[
|   {
|     id: 17222007,
|     name: 'Sweet & cosy room next to Canal Saint Martin ❤️',
|     number_of_reviews: 3067
|   },
|   {
|     id: 26244787,
|     name: 'Double/Twin Room, close to Opera and the Louvre with breakfast included',
|     number_of_reviews: 2620
|   },
|   {
|     id: 41020735,
|     name: 'Bed in Dorm of 8 Beds "The Big One" in Paris',
|     number_of_reviews: 2294
|   },
|   {
|     id: 40194697,
|     name: 'Comfortable bed in shared rooms of 8 in Paris 12e',
|     number_of_reviews: 2105
|   },
|   {
|     id: 35145338,
|     name: 'Nice Room for 2 people',
|     number_of_reviews: 2048
|   }
| ]
```

2-1 Requêtes simples

mongosh

Q3 — Nombre d'hôtes différents

```
noscites> db.listings.distinct("host_id").length  
71979
```

2-1 Requêtes simples

mongosh

Q3 — Nombre d'hôtes différents

```
noscites> db.listings.distinct("host_id").length  
71979
```

Q4 — Réservation instantanée

```
noscites> db.listings.countDocuments({ instant_bookable: "t" })  
22094
```


2-1 Requêtes simples

mongosh

Q3 — Nombre d'hôtes différents

```
noscites> db.listings.distinct("host_id").length  
71979
```

Q4 — Réservation instantanée

```
noscites> db.listings.countDocuments({ instant_bookable: "t" })  
22094
```

Q6 — Super hôtes

```
noscites> db.listings.distinct("host_id", { host_is_superhost: "t" }).length  
10027
```

2-1 Requêtes simples

mongosh

Q3 — Nombre d'hôtes différents

```
noscites> db.listings.distinct("host_id").length  
71979
```

Q4 — Réservation instantanée

```
noscites> db.listings.countDocuments({ instant_bookable: "t" })  
22094
```

Q6 — Super hôtes

```
noscites> db.listings.distinct("host_id", { host_is_superhost: "t" }).length  
10027
```

Q5 — Hôtes avec plus de 100 annonces

```
noscites> db.listings.aggregate([  
  { $group: {  
    _id: "$host_id",  
    host_name: { $first: "$host_name" },  
    nb_annonces: { $max: "$host_total_listings_count" }  
  }},  
  { $match: { nb_annonces: { $gt: 100 } } },  
  { $sort: { nb_annonces: -1 } }  
)]  
  
[  
  {  
    _id: 439074505, host_name: 'Travelnest', nb_annonces: 6278 },  
    {  
      _id: 337698666,  
      host_name: 'Agence COCOONR / BOOK&PAY',  
      nb_annonces: 3256  
    },  
    {  
      _id: 270230184, host_name: 'Janna - BELVILLA', nb_annonces: 1954 },  
    {  
      _id: 400123061, host_name: 'Maeva', nb_annonces: 1665 },  
    {  
      _id: 402191311, host_name: 'GuestReady', nb_annonces: 1631 },  
    {  
      _id: 484535538, host_name: 'RoomPicks', nb_annonces: 1448 },  
    {  
      _id: 33889201, host_name: 'Veeve', nb_annonces: 1024 },  
    {  
      _id: 137510244, host_name: 'Novasol', nb_annonces: 934 },  
    {  
      _id: 50502817, host_name: 'Pierre De WeHost', nb_annonces: 840 },  
    {  
      _id: 314994947, host_name: 'Blueground', nb_annonces: 784 },  
    {  
      _id: 555486771, host_name: 'Holidu', nb_annonces: 699 },  
    {  
      _id: 22805631, host_name: 'Latin Exclusive', nb_annonces: 682 },  
    {  
      _id: 555485516, host_name: 'Holidu', nb_annonces: 652 },  
    {  
      _id: 117238503, host_name: 'Sophie', nb_annonces: 620 },  
    {  
      _id: 337972555, host_name: 'Reservation Desk', nb_annonces: 503 },  
    {  
      _id: 7642792, host_name: 'Ludovic', nb_annonces: 456 },  
    {  
      _id: 460047164, host_name: 'FlexLiving', nb_annonces: 449 },  
    {  
      _id: 137504779, host_name: 'Novasol', nb_annonces: 447 },  
    {  
      _id: 153911376, host_name: 'Novasol', nb_annonces: 446 },  
    {  
      _id: 26981054,  
      host_name: 'Cédric De ClickYourFlat',  
      nb_annonces: 443  
    }  
  ]
```

2-2 Requêtes complexes

Python + Polars

Pourquoi Polars et non pandas ? Polars est 5 à 10x plus rapide que pandas sur les grandes volumétries. Il utilise des types stricts et évite les ambiguïtés de types mixtes fréquentes dans les données Airbnb.

```
moylig@DESKTOP-CUIQHFG:/mnt/c/Users/moymo/OC/P7$ python3 requetes_complexes.py
Connexion à MongoDB réussie.
Récupération des données en cours...

Documents chargés : 95,885
Colonnes          : ['host_is_superhost', 'neighbourhood_cleansed', 'room_type', 'availability_30', 'number_of_reviews']

=====
```

2-2 Requêtes complexes

Python + Polars

Q7 – Taux de réservation moyen par mois par type de logement

shape: (4, 4)

room_type	taux_moyen_%	taux_median_%	nb_annonces
---	---	---	---
str	f64	f64	u32
Entire home/apt	71.29	86.67	85733
Private room	70.29	93.33	8975
Shared room	60.72	73.33	401
Hotel room	53.53	50.0	776

2-2 Requêtes complexes

Python + Polars

Q7 – Taux de réservation moyen par mois par type de logement

shape: (4, 4)

room_type	taux_moyen_%	taux_median_%	nb_annonces
---	---	---	---
str	f64	f64	u32
Entire home/apt	71.29	86.67	85733
Private room	70.29	93.33	8975
Shared room	60.72	73.33	401
Hotel room	53.53	50.0	776

Q8 – Médiane des nombre d'avis (tous logements)

Médiane : 3.0
Moyenne : 19.89
Maximum : 3067

La médiane à 3 signifie que 50% des logements ont 3 avis ou moins - beaucoup d'annonces très peu actives.

2-2 Requêtes complexes

Python + Polars

Q7 – Taux de réservation moyen par mois par type de logement

shape: (4, 4)

room_type	taux_moyen_%	taux_median_%	nb_annonces
---	---	---	---
str	f64	f64	u32
Entire home/apt	71.29	86.67	85733
Private room	70.29	93.33	8975
Shared room	60.72	73.33	401
Hotel room	53.53	50.0	776

Q8 – Médiane des nombre d'avis (tous logements)

Médiane : 3.0
Moyenne : 19.89
Maximum : 3067

La médiane à 3 signifie que 50% des logements ont 3 avis ou moins - beaucoup d'annonces très peu actives.

Q9 – Médiane des avis par catégorie d'hôte

shape: (3, 4)

categorie	mediane_avis	moyenne_avis	nb_logements
---	---	---	---
str	f64	f64	u32
Super hôte	24.0	50.7	14776
	12.5	29.08	74
Hôte classique	2.0	14.26	81035

Les super hôtes ont une médiane 12x plus élevée - ils sont beaucoup plus actifs et génèrent plus de réservations

2-2 Requêtes complexes

Python + Polars

Q10 – Top 20 quartiers par densité de logements

shape: (20, 2)

neighbourhood_cleansed	nb_logements
---	---
str	u32
Buttes-Montmartre	10555
Popincourt	8430
Vaugirard	7802
Batignolles-Monceau	6857
Entrepôt	6558
...	...
Élysée	2898
Hôtel-de-Ville	2821
Palais-Bourbon	2740
Luxembourg	2701
Louvre	2026

Q11 – Top 20 quartiers par taux de réservation moyen

shape: (20, 3)

neighbourhood_cleansed	taux_moyen_%	nb_logements
---	---	---
str	f64	u32
Ménilmontant	75.42	5271
Entrepôt	74.81	6558
Popincourt	74.78	8430
Buttes-Chaumont	74.13	5465
Panthéon	73.14	3010
...	...	...
Palais-Bourbon	68.87	2740
Bourse	68.69	3038
Luxembourg	66.45	2701
Élysée	62.45	2898
Passy	62.3	6407

02

Mise à disposition BI (Business Intelligence)

OBJECTIF : Connecter MongoDB à un outil BI pour visualiser les données Airbnb

Mise à disposition BI

Étape 1 — Export CSV

```
moylig@DESKTOP-CUIQHF6:/mnt/c/Users/moymo/OC/P7$ mongoexport \
--port 27029 \
--db noscites \
--collection listings \
--type csv \
--fields id,name,city,neighbourhood_cleansed,room_type,price,availability_30,availability_365,number_of_reviews,host_is_superhost,instant_bookable,review_scores_rating,accommodat
es,beds \
es,beds \ Open file in editor (ctrl + click)
--out "/mnt/c/Users/moymo/OC/P7/noscites_tableau.csv"
2026-03-17T20:06:47.560+0100 connected to: mongodb://localhost:27029/
2026-03-17T20:06:48.561+0100 [#####.....] noscites.listings 32000/105858 (30.2%)
2026-03-17T20:06:49.562+0100 [#####.....] noscites.listings 56000/105858 (52.9%)
2026-03-17T20:06:50.071+0100 [#####.....] noscites.listings 105858/105858 (100.0%)
2026-03-17T20:06:50.071+0100 exported 105858 records
```

Mise à disposition BI

Étape 2 — Import dans Tableau

Mise à disposition BI

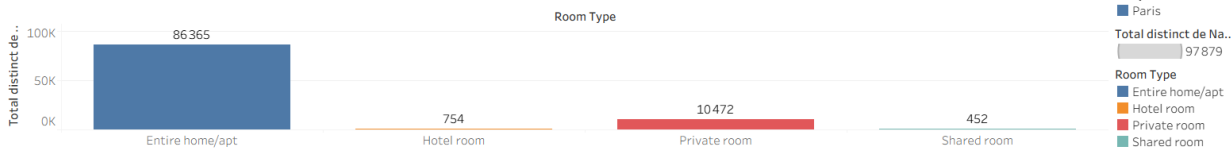
Étape 3 — Visualisations créées dans Tableau

Analyse Airbnb — NosCités | Paris & Lyon

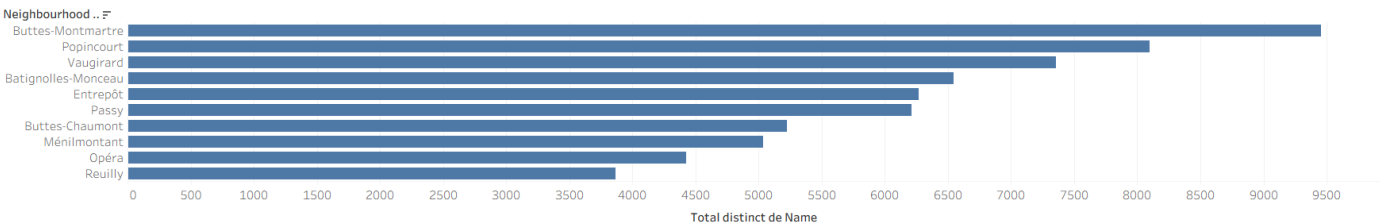
Paris vs Lyon



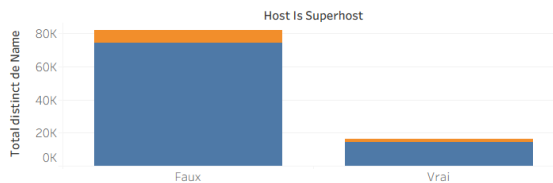
Types de logements



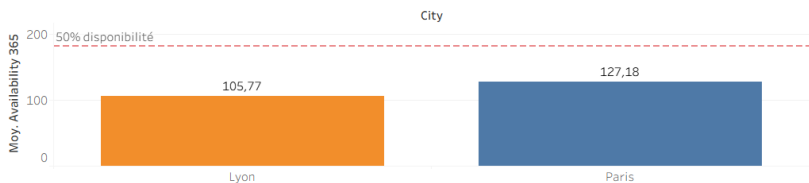
Top 10 Quartiers Paris



Superhôtés



Disponibilité moyenne



03

Architecture haute disponibilité

3-1 Schéma de la base de données MongoDB

STRUCTURE DU DOCUMENT listings (75 champs)

- **Identifiant** : `_id` (ObjectId), `id` (Int64), `city` (String → clé de shard)
- **Hôte** : `host_id`, `host_name`, `host_is_superhost` ("t"|"f"), `host_listings_count`
- **Localisation** : `neighbourhood_cleansed`, `latitude`, `longitude`, `city`
- **Logement** : `room_type`, `property_type`, `price` (String), `accommodates`, `bathrooms`
- **Disponibilité** : `availability_30/60/90/365` (Int32), `has_availability` ("t"|"f")
- **Avis** : `number_of_reviews`, `reviews_per_month`, `review_scores_rating`
- **Semi-structurés** : `amenities` (Array JSON), `description` (String long), `host_verifications` (Array)

3-2 Fusion des collections

Python + pandas

Fusion Paris + Lyon. On commence par taguer Lyon avec le champ `city`, puis on importe Lyon dans la même collection

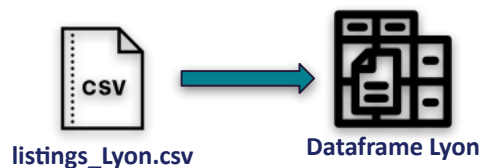


`listings_Lyon.csv`

3-2 Fusion des collections

Python + pandas

Fusion Paris + Lyon. On commence par taguer Lyon avec le champ `city`, puis on importe Lyon dans la même collection



3-2 Fusion des collections

Python + pandas

Fusion Paris + Lyon. On commence par taguer Lyon avec le champ `city`, puis on importe Lyon dans la même collection



3-2 Fusion des collections

Python + pandas

Fusion Paris + Lyon. On commence par taguer Lyon avec le champ `city`, puis on importe Lyon dans la même collection



3-2 Fusion des collections

Python + pandas

Fusion Paris + Lyon. On commence par taguer Lyon avec le champ `city`, puis on importe Lyon dans la même collection



3-2 Fusion des collections

Python + pandas

Fusion Paris + Lyon. On commence par taguer Lyon avec le champ `city`, puis on importe Lyon dans la même collection



```
moylig@DESKTOP-CUIQHFG:/mnt/c/Users/moymo/OC/P7$ python3 -c "  
import pandas as pd  
df = pd.read_csv('listings_Lyon.csv')  
df['city'] = 'Lyon'  
df.to_csv('listings_Lyon_tagged.csv', index=False)  
print(f'Lyon taggé : {len(df)} documents')  
"  
Lyon taggé : 9973 documents
```

```
moylig@DESKTOP-CUIQHFG:/mnt/c/Users/moymo/OC/P7$ mongoimport \  
--db noscites \  
--collection listings \  
--type csv \  
--headerline \  
--file "listings_Lyon_tagged.csv"  
2026-03-16T18:12:23.948+0100    connected to: mongodb://localhost/  
2026-03-16T18:12:26.476+0100    9973 document(s) imported successfully. 0 document(s) failed to import.
```

3-2 Fusion des collections

mongosh

Ajout du champ **city** aux documents Paris existants



3-2 Fusion des collections

mongosh

Ajout du champ **city** aux documents Paris existants



3-2 Fusion des collections

mongosh

Ajout du champ **city** aux documents Paris existants



```
noscites> db.listings.updateMany(  
|   { city: { $exists: false } },  
|   { $set: { city: "Paris" } }  
| )  
|  
| {  
|   acknowledged: true,  
|   insertedId: null,  
|   matchedCount: 95885,  
|   modifiedCount: 95885,  
|   upsertedCount: 0  
| }
```

3-2 Fusion des collections

mongosh

Ajout du champ **city** aux documents Paris existants



Si le champs **city** n'existe pas,
ajout d'un champs **city**,
avec comme valeur **Paris**



```
noscites> db.listings.updateMany(  
  { city: { $exists: false } },  
  { $set: { city: "Paris" } }  
)  
{  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 95885,  
  modifiedCount: 95885,  
  upsertedCount: 0  
}
```

Vérifications

```
noscites> db.listings.countDocuments({ city: "Paris" })  
95885  
noscites> db.listings.countDocuments({ city: "Lyon" })  
9973  
noscites> db.listings.countDocuments()  
105858
```

3-2 Fusion des collections

mongosh

Ajout du champ **city** aux documents Paris existants



```
noscites> db.listings.updateMany(  
  { city: { $exists: false } },  
  { $set: { city: "Paris" } }  
)  
{  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 95885,  
  modifiedCount: 95885,  
  upsertedCount: 0  
}
```

Vérifications

```
noscites> db.listings.countDocuments({ city: "Paris" })  
95885  
noscites> db.listings.countDocuments({ city: "Lyon" })  
9973  
noscites> db.listings.countDocuments()  
105858
```

Les données Paris et Lyon ont été importées dans une unique collection `noscites.listings`.

Le champ **city** permet de les distinguer et constitue la base du sharding géographique

3-3 Réplication : ReplicaSet MongoDB

ReplicaSet MongoDB — Architecture & Rôles

Un ReplicaSet est un groupe de processus MongoDB qui maintiennent le même jeu de données.

PRIMARY :27017

rs0 | Priorité 2

Toutes les écritures
Lecture activée

Oplog → diffuse
les changements

SECONDARY 1 : 27018

rs1 | Priorité 1

Réplique via oplog
Lecture seule
← réplication oplog →
Peut devenir
PRIMARY

SECONDARY 2 : 27019

rs2 | Priorité 1

Réplique via oplog
Lecture seule
← réplication oplog →
Peut devenir
PRIMARY

ARBITRE : 27020

rs3 | Priorité 0

Vote uniquement
Pas de données
- - - vote seulement - - -
Départage les votes
en cas d'élection

Quorum (principe RAFT) :

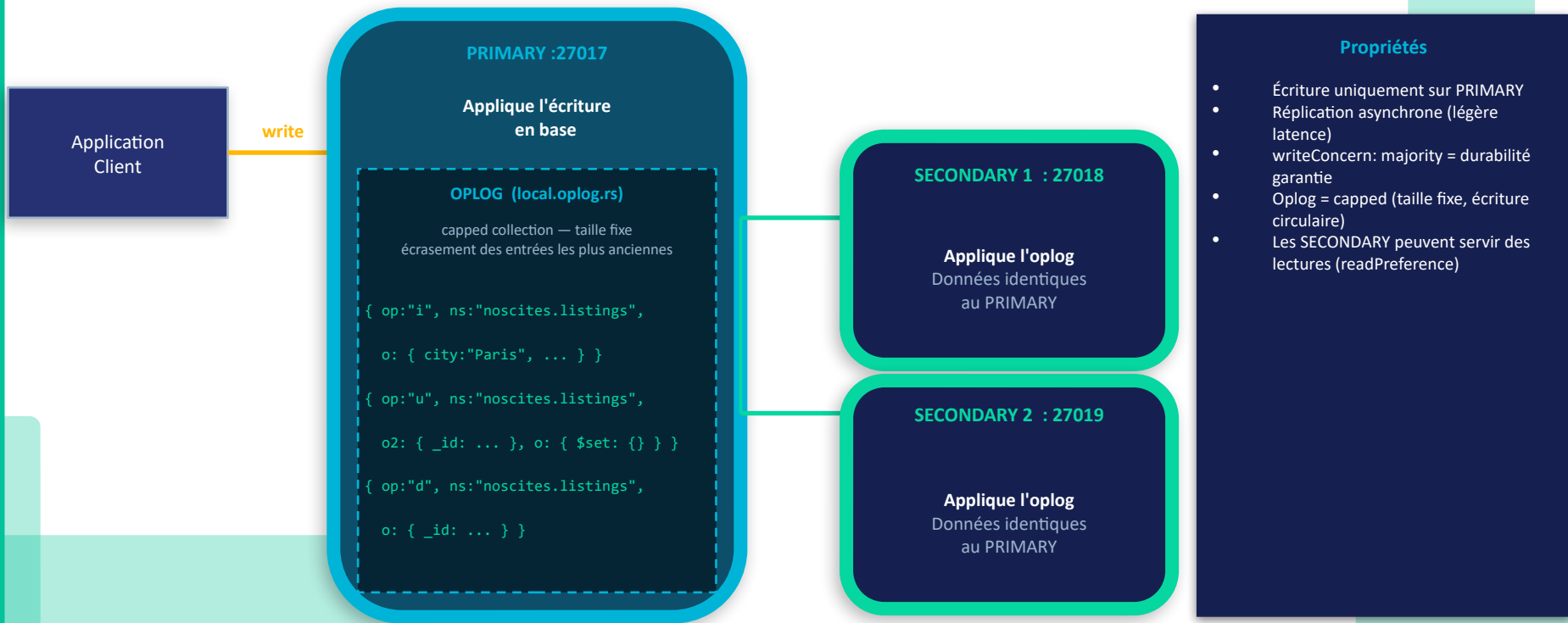
∅ 4 membres | **3 membres votants** (PRIMARY vote, les 2 SECONDARY + ARBITRE) | **Majorité = 2 votes**

→ Le cluster tolère la panne d'**1 membre votant** avant de perdre le quorum et passer en lecture seule.

3-3 Réplication : ReplicaSet MongoDB

Réplication — L'Oplog, journal des opérations

Chaque écriture sur le PRIMARY est enregistrée dans l'oplog, puis propagée aux SECONDARY



3-3 Réplication : ReplicaSet MongoDB

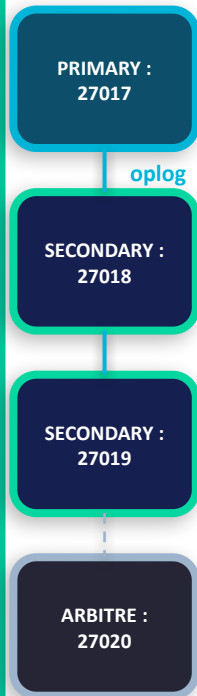
Scénario : panne du PRIMARY → élection automatique en ~10-30 secondes

Élection — Quorum & Tolérance aux pannes

3-3 Réplication : ReplicaSet MongoDB

Scénario : panne du PRIMARY → élection automatique en ~10-30 secondes

1 État normal



Élection — Quorum & Tolérance aux pannes

3-3 Réplication : ReplicaSet MongoDB

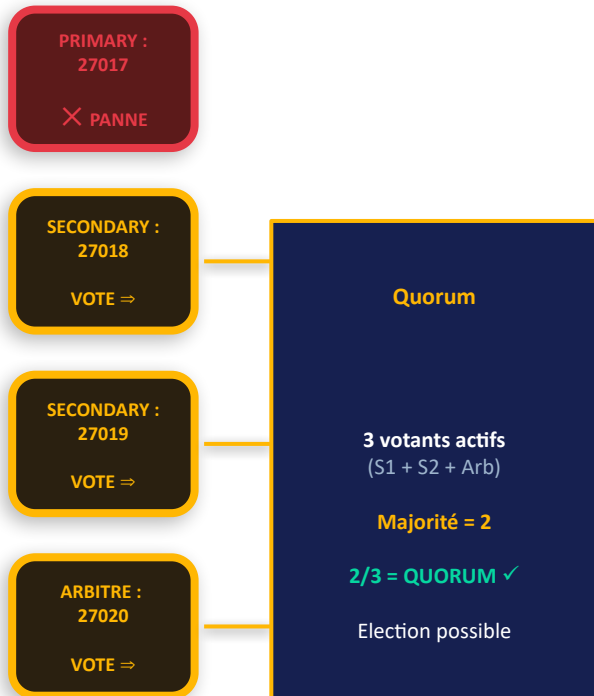
Scénario : panne du PRIMARY → élection automatique en ~10-30 secondes

Élection — Quorum & Tolérance aux pannes

1 État normal



2 Panne PRIMARY



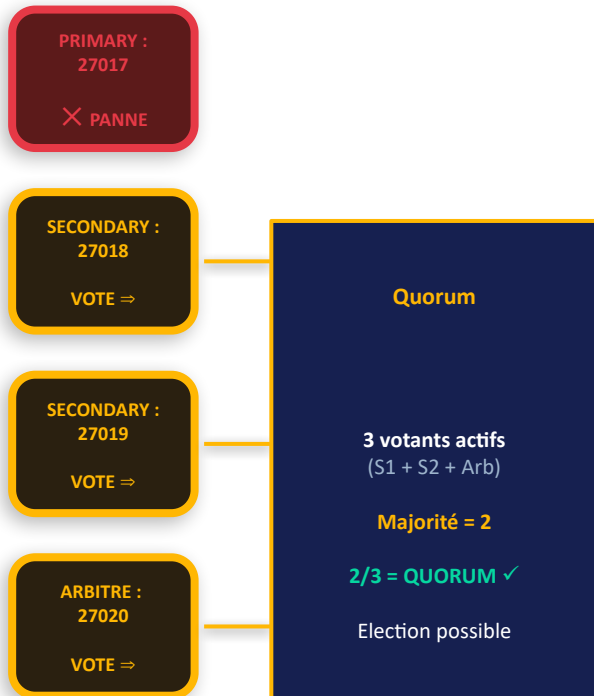
3-3 Réplication : ReplicaSet MongoDB

Scénario : panne du PRIMARY → élection automatique en ~10-30 secondes

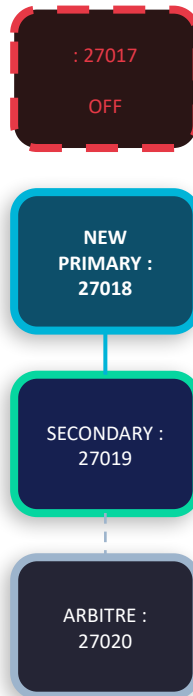
1 État normal



2 Panne PRIMARY



3 Après élection



Résultat de l'élection

Durée élection : ~10-30 sec

- 0 perte de données — oplog complet sur chaque SECONDARY
- Le client se reconnecte automatiquement au nouveau PRIMARY
- Quand :27017 revient → redevient SECONDARY (plus bas rang de priorité)
- Applications transparentes si driver MongoDB configuré correctement

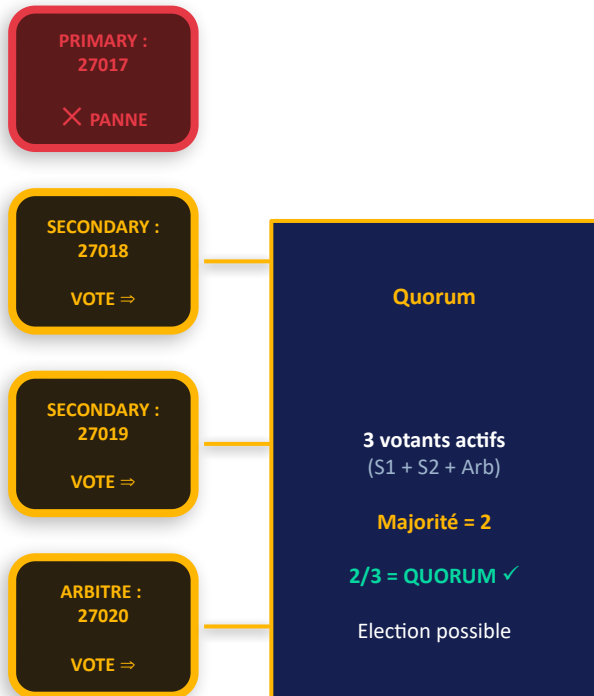
3-3 Réplication : ReplicaSet MongoDB

Scénario : panne du PRIMARY → élection automatique en ~10-30 secondes

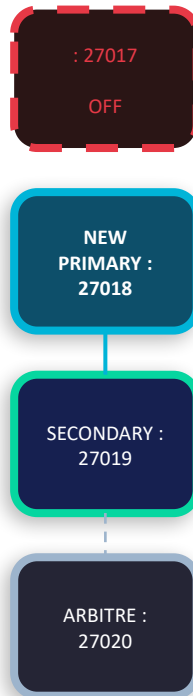
1 État normal



2 Panne PRIMARY



3 Après élection



Résultat de l'élection

Durée élection : ~10-30 sec

- 0 perte de données — oplog complet sur chaque SECONDARY
- Le client se reconnecte automatiquement au nouveau PRIMARY
- Quand :27017 revient → redevient SECONDARY (plus bas rang de priorité)
- Applications transparentes si driver MongoDB configuré correctement

3-3 Réplication : ReplicaSet MongoDB

Mise en place du ReplicaSet —
Séquence de démarrage

3-3 Réplication : ReplicaSet MongoDB

1

STOP

Arrêt
standalone

Aucune instance
active

Commandes

```
mongod --shutdown  
--dbpath ~/mongodb/standalone
```



Standalone

Mise en place du ReplicaSet —
Séquence de démarrage

3-3 Réplication : ReplicaSet MongoDB

Mise en place du ReplicaSet —
Séquence de démarrage

1

STOP

Arrêt
standalone

Aucune instance
active

Commandes

```
mongod --shutdown  
--dbpath ~/mongodb/standalone
```

```
moylig@DESKTOP-CUIQHFG:/mnt/c/Users/moymo/OC/P7$ mongosh --eval 'db.adminCommand({ shutdown: 1 })' --port 27017  
MongoNetworkError: connection 4 to 127.0.0.1:27017 closed
```



Standalone

3-3 Réplication : ReplicaSet MongoDB

1

STOP

Arrêt
standalone

Aucune instance
active

Commandes

```
mongod --shutdown  
--dbpath ~/mongodb/standalone
```



Standalone

Mise en place du ReplicaSet —
Séquence de démarrage

3-3 Réplication : ReplicaSet MongoDB

Mise en place du ReplicaSet —
Séquence de démarrage

1

STOP

Arrêt
standalone

Aucune instance
active

2

DIRS

Création
répertoires

4 dossiers de
données créés

Commandes

```
mongod --shutdown  
--dbpath ~/mongodb/standalone
```

```
mkdir -p ~/mongodb/rs{0,1,2,3}
```



Standalone



Dirs créés

3-3 Réplication : ReplicaSet MongoDB

Mise en place du ReplicaSet —
Séquence de démarrage

1

STOP

Arrêt
standalone

Aucune instance
active

2

DIRS

Création
répertoires

4 dossiers de
données créés

Commandes

```
mongod --shutdown  
--dbpath ~/mongodb/standalone
```

```
mkdir -p ~/mongodb/rs{0,1,2,3}
```

```
mkdir -p ~/mongodb/rs0 ~/mongodb/rs1 ~/mongodb/rs2 ~/mongodb/rs3
```



Standalone



Dirs créés

3-3 Réplication : ReplicaSet MongoDB

Mise en place du ReplicaSet —
Séquence de démarrage

1

STOP

Arrêt
standalone

Aucune instance
active

2

DIRS

Création
répertoires

4 dossiers de
données créés

Commandes

```
mongod --shutdown  
--dbpath ~/mongodb/standalone
```

```
mkdir -p ~/mongodb/rs{0,1,2,3}
```



Standalone



Dirs créés

3-3 Réplication : ReplicaSet MongoDB

Mise en place du ReplicaSet —
Séquence de démarrage

1

STOP

Arrêt
standalone

Aucune instance
active

2

DIRS

Création
répertoires

4 dossiers de
données créés

3

RUN

Démarrage
4 instances

4 processus mongod
isolés — pas encore
configurés en RS

Commandes

```
mongod --shutdown  
--dbpath ~/mongodb/standalone
```

```
mkdir -p ~/mongodb/rs{0,1,2,3}
```

```
mongod --replSet noscitesRS  
--port 27017 --dbpath rs0  
--port 27018 --dbpath rs1  
...
```



Standalone



Dirs créés



Processus démarrés

3-3 Réplication : ReplicaSet MongoDB

Mise en place du ReplicaSet —
Séquence de démarrage

1

STOP

Arrêt
standalone

Aucune instance
active

2

DIRS

Création
répertoires

4 dossiers de
données créés

3

RUN

Démarrage
4 instances

4 processus mongod
isolés — pas encore
configurés en RS

```
moylig@DESKTOP-CUIQHFG:/mnt/c/Users/moymo/OC/P7$ mongod --replSet noscitesRS --port 27017 --dbpath ~/mongodb/rs0 --fork --logpath ~/mongodb/rs0/mongod.log --bind_ip_all
about to fork child process, waiting until server is ready for connections.
forked process: 710
child process started successfully, parent exiting
moylig@DESKTOP-CUIQHFG:/mnt/c/Users/moymo/OC/P7$ mongod --replSet noscitesRS --port 27018 --dbpath ~/mongodb/rs1 --fork --logpath ~/mongodb/rs1/mongod.log --bind_ip_all
about to fork child process, waiting until server is ready for connections.
forked process: 818
child process started successfully, parent exiting
moylig@DESKTOP-CUIQHFG:/mnt/c/Users/moymo/OC/P7$ mongod --replSet noscitesRS --port 27019 --dbpath ~/mongodb/rs2 --fork --logpath ~/mongodb/rs2/mongod.log --bind_ip_all
about to fork child process, waiting until server is ready for connections.
forked process: 924
child process started successfully, parent exiting
moylig@DESKTOP-CUIQHFG:/mnt/c/Users/moymo/OC/P7$ mongod --replSet noscitesRS --port 27020 --dbpath ~/mongodb/rs3 --fork --logpath ~/mongodb/rs3/mongod.log --bind_ip_all
about to fork child process, waiting until server is ready for connections.
forked process: 1085
child process started successfully, parent exiting
```

Standalone

Dirs créés

Processus démarrés

3-3 Réplication : ReplicaSet MongoDB

Mise en place du ReplicaSet —
Séquence de démarrage

1

STOP

Arrêt
standalone

Aucune instance
active

2

DIRS

Création
répertoires

4 dossiers de
données créés

3

RUN

Démarrage
4 instances

4 processus mongod
isolés — pas encore
configurés en RS

Commandes

```
mongod --shutdown  
--dbpath ~/mongodb/standalone
```

```
mkdir -p ~/mongodb/rs{0,1,2,3}
```

```
mongod --replSet noscitesRS  
--port 27017 --dbpath rs0  
--port 27018 --dbpath rs1  
...
```



Standalone



Dirs créés



Processus démarrés

3-3 Réplication : ReplicaSet MongoDB

Mise en place du ReplicaSet —
Séquence de démarrage

1

STOP

Arrêt
standalone

Aucune instance
active

2

DIRS

Création
répertoires

4 dossiers de
données créés

3

RUN

Démarrage
4 instances

4 processus mongod
isolés — pas encore
configurés en RS

4

INIT

Initialisation
ReplicaSet

Élection PRIMARY
rs0 → Réplication
oplog activée

Commandes

```
mongod --shutdown  
--dbpath ~/mongodb/standalone
```

```
mkdir -p ~/mongodb/rs{0,1,2,3}
```

```
mongod --replSet nscitesRS  
--port 27017 --dbpath rs0  
--port 27018 --dbpath rs1  
...
```

```
rs.initiate()  
rs.add('27018')  
rs.add('27019')  
rs.addArb('27020')
```



Standalone



Dirs créés



Processus démarrés



RS initialisé

3-3 Réplication : ReplicaSet MongoDB

Mise en place du ReplicaSet — Séquence de démarrage

1

STOP

Arrêt
standalone

Aucune instance
active

2

DIRS

Création
répertoires

4 dossiers de
données créés

3

RUN

Démarrage
4 instances

4 processus mongod
isolés — pas encore
configurés en RS

4

INIT

Initialisation
ReplicaSet

Élection PRIMARY
rs0 → Réplication
oplog activée

Commandes

```
mongod --shutdown  
--dbpath ~/mongodb/standalone
```

```
mkdir -p ~/mongodb/rs{0,1,2,3}
```

```
mongod --replSet noscitesRS  
--port 27017 --dbpath rs0  
--port 27018 --dbpath rs1  
...
```

```
test> rs.initiate({  
  _id: "noscitesRS",  
  members: [  
    { _id: 0, host: "localhost:27017", priority: 2 },  
    { _id: 1, host: "localhost:27018", priority: 1 },  
    { _id: 2, host: "localhost:27019", priority: 1 },  
    { _id: 3, host: "localhost:27020", arbiterOnly: true }  
  ]  
})  
{  
  ok: 1,  
  '$clusterTime': {  
    clusterTime: Timestamp({ t: 1773684404, i: 1 }),  
    signature: {  
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA', 0),  
      keyId: Long('0')  
    }  
  },  
  operationTime: Timestamp({ t: 1773684404, i: 1 })  
}
```



Standalone



Dirs créés



Processus démarrés

3-3 Réplication : ReplicaSet MongoDB

Mise en place du ReplicaSet —
Séquence de démarrage

1

STOP

Arrêt
standalone

Aucune instance
active

2

DIRS

Création
répertoires

4 dossiers de
données créés

3

RUN

Démarrage
4 instances

4 processus mongod
isolés — pas encore
configurés en RS

4

INIT

Initialisation
ReplicaSet

Élection PRIMARY
rs0 → Réplication
oplog activée

Commandes

```
mongod --shutdown  
--dbpath ~/mongodb/standalone
```

```
mkdir -p ~/mongodb/rs{0,1,2,3}
```

```
mongod --replSet nscitesRS  
--port 27017 --dbpath rs0  
--port 27018 --dbpath rs1  
...
```

```
rs.initiate()  
rs.add(':27018')  
rs.add(':27019')  
rs.addArb(':27020')
```



Standalone



Dirs créés



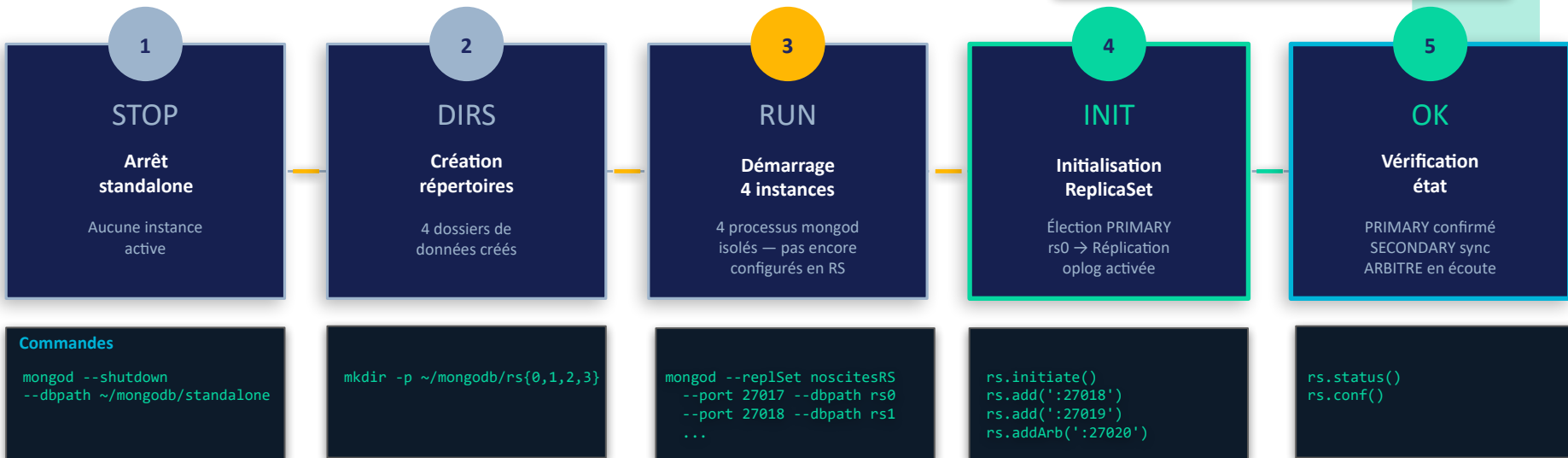
Processus démarrés



RS initialisé

3-3 Réplication : ReplicaSet MongoDB

Mise en place du ReplicaSet — Séquence de démarrage



Standalone



Dirs créés



Processus démarrés



RS initialisé



✓ Opérationnel

3-3 Réplication : ReplicaSet MongoDB

Mise en place du ReplicaSet — Séquence de démarrage

1

STOP

Arrêt
standalone

Aucune instance
active

2

DIRS

Création
répertoires

4 dossiers de
données créés

3

RUN

Démarrage
4 instances

4 processus mongod
isolés — pas encore
configurés en RS

4

INIT

Initialisation
ReplicaSet

Élection PRIMARY
rs0 → Réplication
oplog activée

5

OK

Vérification
état

PRIMARY confirmé
SECONDARY sync
ARBITER en écoute

Commandes

```
mongod --shutdown  
--dbpath ~/mongodb/standalone
```

```
mkdir -p ~/mongodb/rs{0,1,2,3}
```

```
mongod --replSet nscitesRS  
--port 27017 --dbpath rs0  
--port 27018 --dbpath rs1  
...
```

```
rs.initiate()  
rs.add('27018')  
rs.add('27019')  
rs.addArb('27020')
```

```
rs.status()  
rs.conf()
```

```
moylig@DESKTOP-CUIQHFG:/mnt/c/Users/moymo/OC/P7$ mongosh --port 27017 --eval 'rs.status().members.forEach(m => print(m.name, m.stateStr))'  
localhost:27017 PRIMARY  
localhost:27018 SECONDARY  
localhost:27019 SECONDARY  
localhost:27020 ARBITER
```

3-3 Réplication : ReplicaSet MongoDB

Mise en place du ReplicaSet — Séquence de démarrage

1

STOP

**Arrêt
standalone**

Aucune instance
active

2

DIRS

**Création
répertoires**

4 dossiers de
données créés

3

RUN

**Démarrage
4 instances**

4 processus mongod
isolés — pas encore
configurés en RS

4

INIT

**Initialisation
ReplicaSet**

Élection PRIMARY
rs0 → Réplication
oplog activée

5

OK

**Vérification
état**

PRIMARY confirmé
SECONDARY sync
ARBITRE en écoute

Commandes

```
mongod --shutdown  
--dbpath ~/mongodb/standalone
```

```
mkdir -p ~/mongodb/rs{0,1,2,3}
```

```
mongod --replSet nscitesRS  
--port 27017 --dbpath rs0  
--port 27018 --dbpath rs1  
...
```

```
rs.initiate()  
rs.add(':27018')  
rs.add(':27019')  
rs.addArb(':27020')
```

```
rs.status()  
rs.conf()
```



Standalone



Dirs créés



Processus démarrés



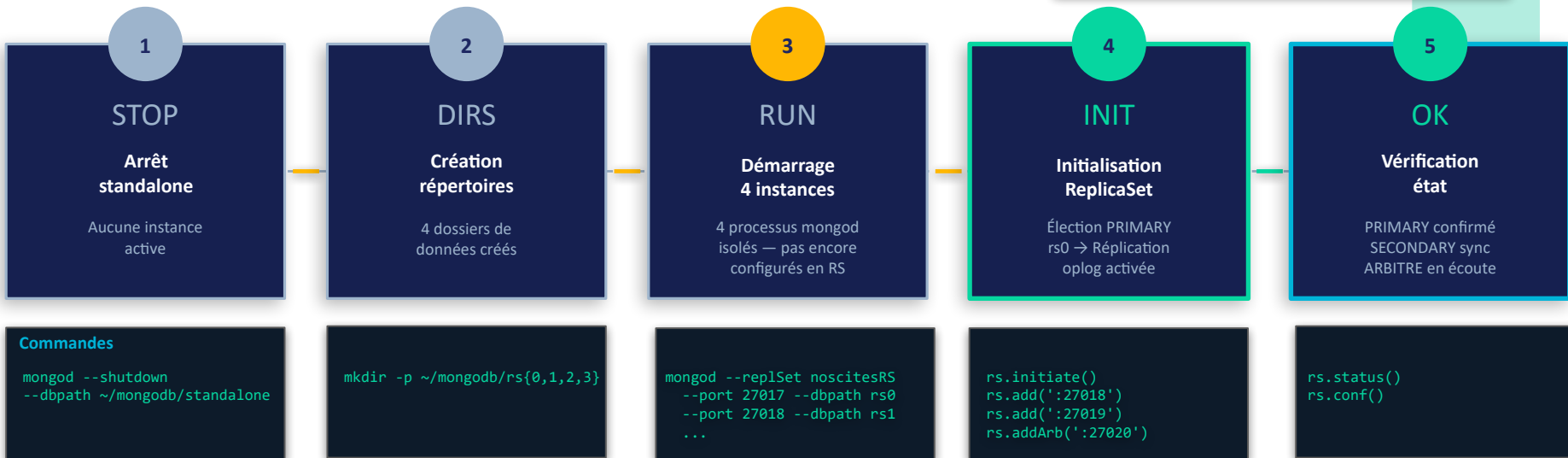
RS initialisé



✓ Opérationnel

3-3 Réplication : ReplicaSet MongoDB

Mise en place du ReplicaSet — Séquence de démarrage



Standalone



Dirs créés



Processus démarrés



RS initialisé



✓ Opérationnel

Ordre critique : les 4 instances mongod doivent être démarrées **avant** rs.initiate(). Le nœud rs0 (port 27017) devient PRIMARY car il exécute l'initiation — les autres entrent en STARTUP2 puis SECONDARY.

3-3 Réplication : ReplicaSet MongoDB

bash

Vérification que les 105 858 documents sont bien répliqués sur les deux SECONDARY

```
moylig@DESKTOP-CUIQHFQ:/mnt/c/Users/moymo/OC/P7$ mongosh --port 27018 --eval '
db.getMongo().setReadPref("secondary");
use("noscites");
print("SECONDARY 1 - Total:", db.listings.countDocuments());
print("Paris:", db.listings.countDocuments({ city: "Paris" }));
print("Lyon:", db.listings.countDocuments({ city: "Lyon" }));
'
SECONDARY 1 - Total: 105858
Paris: 95885
Lyon: 9973
```

```
moylig@DESKTOP-CUIQHFQ:/mnt/c/Users/moymo/OC/P7$ mongosh --port 27019 --eval '
db.getMongo().setReadPref("secondary");
use("noscites");
print("SECONDARY 2 - Total:", db.listings.countDocuments());
print("Paris:", db.listings.countDocuments({ city: "Paris" }));
print("Lyon:", db.listings.countDocuments({ city: "Lyon" }));
'
SECONDARY 2 - Total: 105858
Paris: 95885
Lyon: 9973
```

3-3 Réplication : ReplicaSet MongoDB

Rôle	Port / Dossier	Rôle dans le cluster	Résultat observé
PRIMARY	27017 / ~/mongodb/rs0	Reçoit toutes les écritures. Priorité 2 (élu en premier)	✅ state=1, health=1. 105 858 docs
SECONDARY	27018 / ~/mongodb/rs1	Réplique en temps réel via oplog. Prend le relais si PRIMARY tombe	✅ state=2, health=1. 105 858 docs répliqués
SECONDARY 2	27019 / ~/mongodb/rs2	Réplique en temps réel via oplog. Redondance supplémentaire	✅ state=2, health=1. 105 858 docs répliqués
ARBITRE	27020 / ~/mongodb/rs3	Vote uniquement. Ne stocke pas de données. Garantit le quorum (3/4)	✅ state=7, health=1. Quorum assuré

✅ **Réplication confirmée** : Toute écriture sur le PRIMARY est propagée automatiquement au SECONDARY. En cas de panne du PRIMARY, le SECONDARY est promu automatiquement grâce au vote de l'ARBITRE. Le système reste opérationnel sans intervention manuelle.

3-4 Distribution : Sharding par zone géographique

Découpage horizontal : chaque shard contient un sous-ensemble exclusif des données

Chunks & Clé de
Shard

3-4 Distribution : Sharding par zone géographique

Découpage horizontal : chaque shard contient un sous-ensemble exclusif des données

Chunks & Clé de
Shard

Collection noscites.listings

```
{ city: "Paris", _id: 0001 }
```

```
{ city: "Paris", _id: 0002 }
```

```
{ city: "Lyon", _id: 0003 }
```

```
{ city: "Paris", _id: 0004 }
```

```
{ city: "Lyon", _id: 0005 }
```

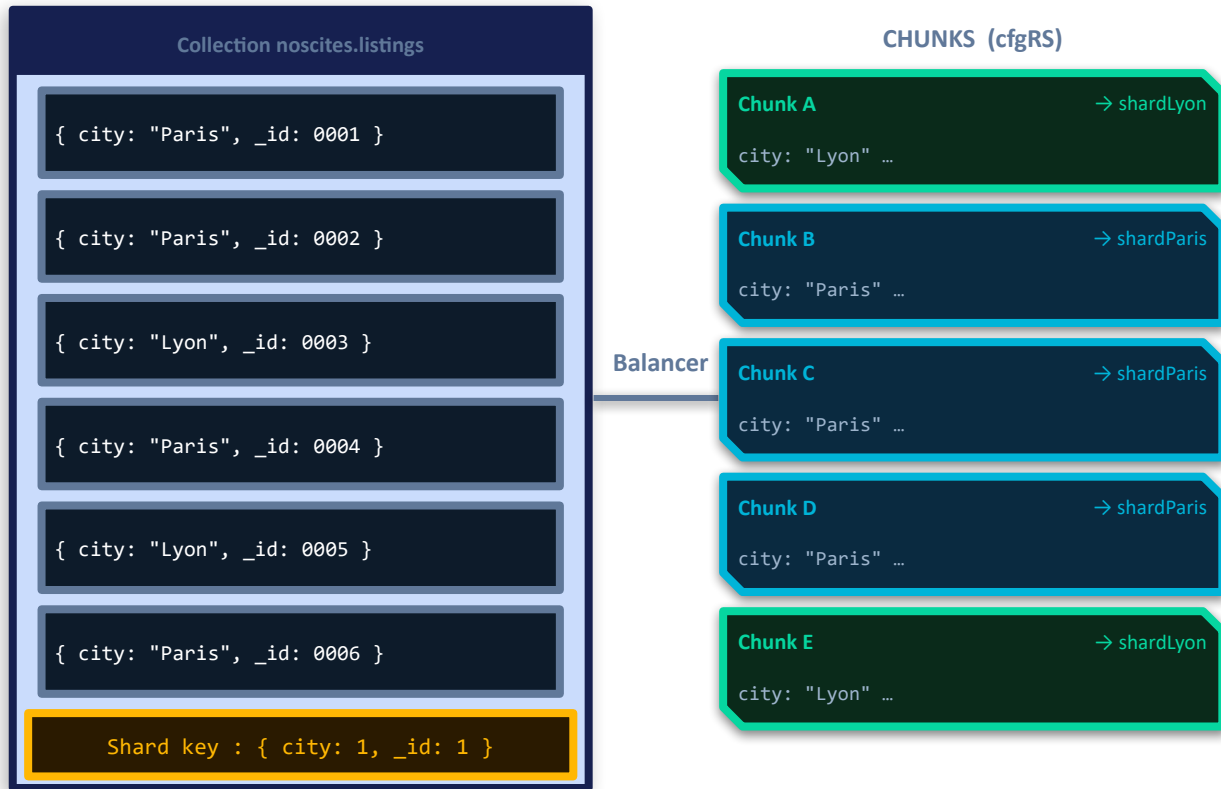
```
{ city: "Paris", _id: 0006 }
```

```
Shard key : { city: 1, _id: 1 }
```

3-4 Distribution : Sharding par zone géographique

Découpage horizontal : chaque shard contient un sous-ensemble exclusif des données

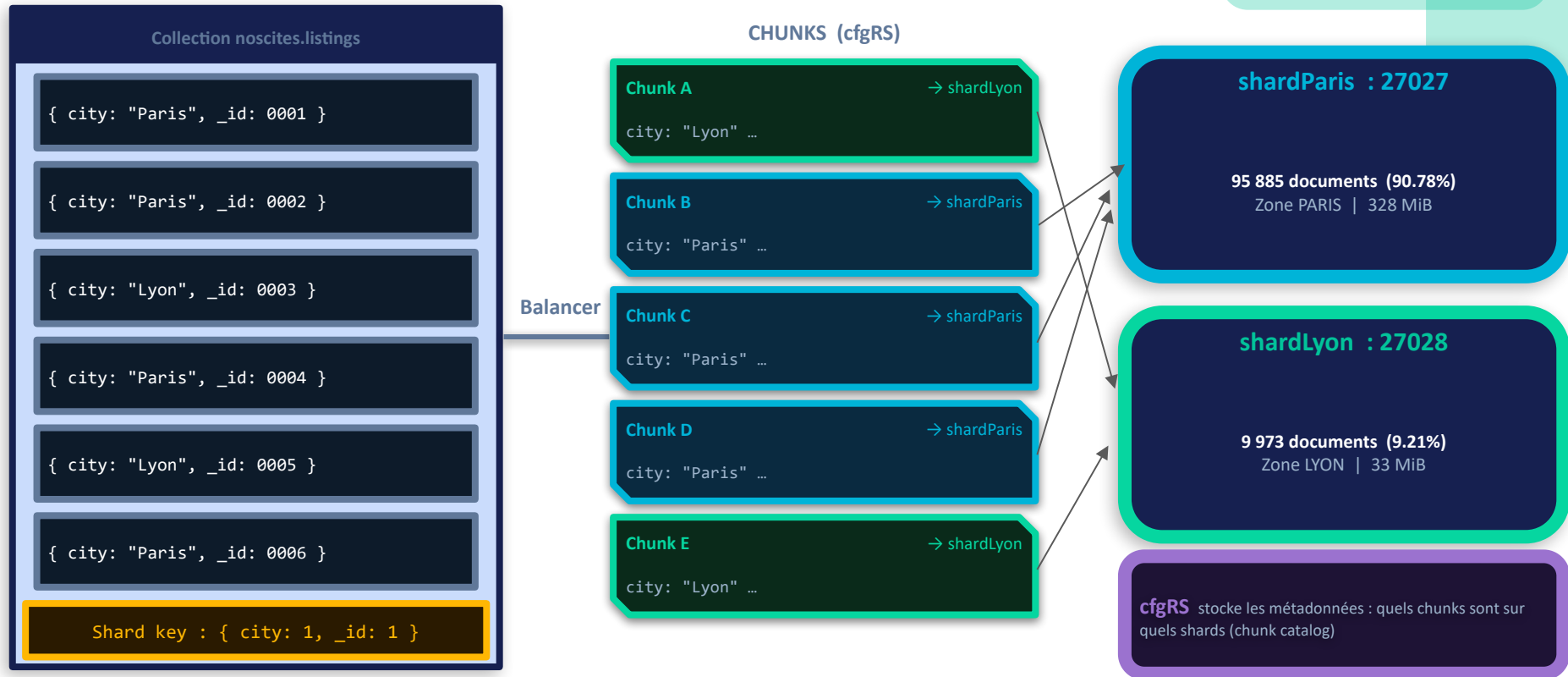
Chunks & Clé de
Shard



3-4 Distribution : Sharding par zone géographique

Découpage horizontal : chaque shard contient un sous-ensemble exclusif des données

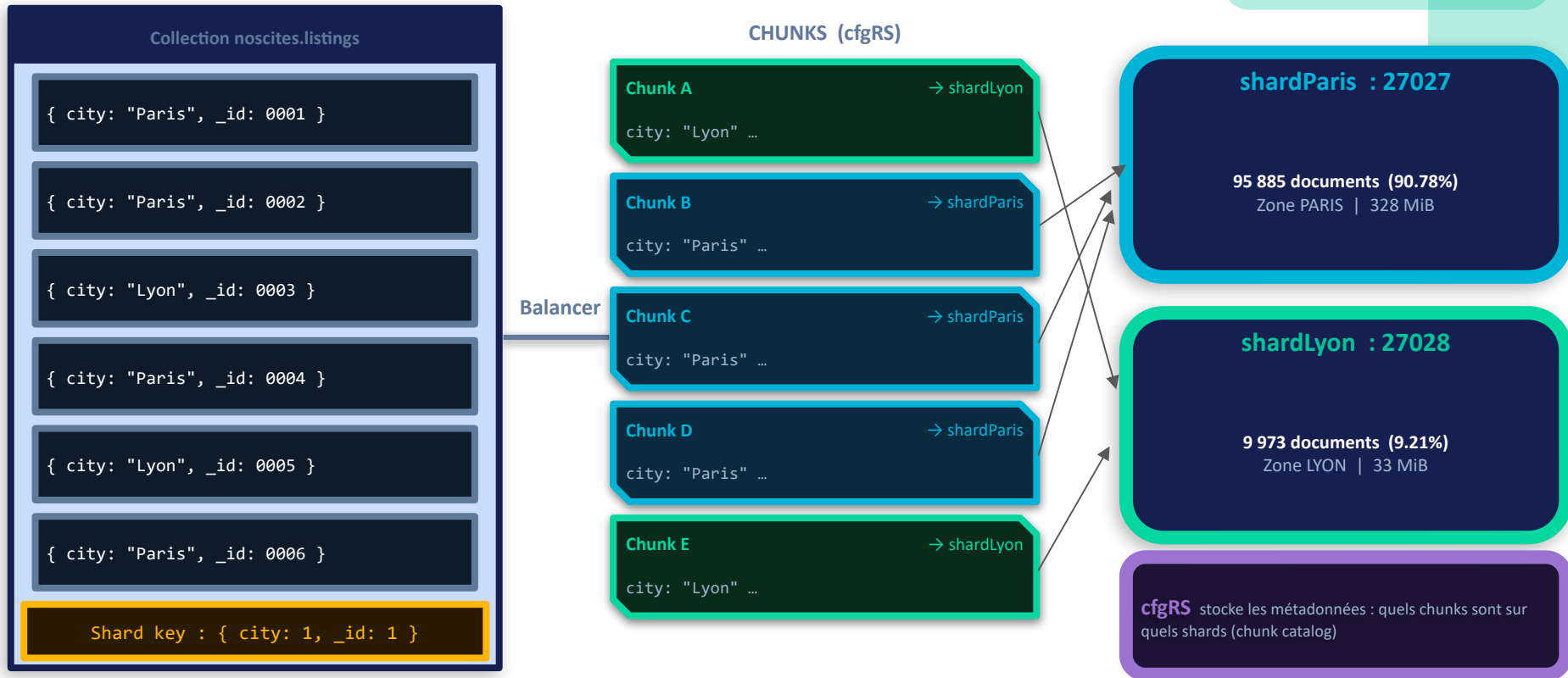
Chunks & Clé de
Shard



3-4 Distribution : Sharding par zone géographique

Découpage horizontal : chaque shard contient un sous-ensemble exclusif des données

Chunks & Clé de Shard

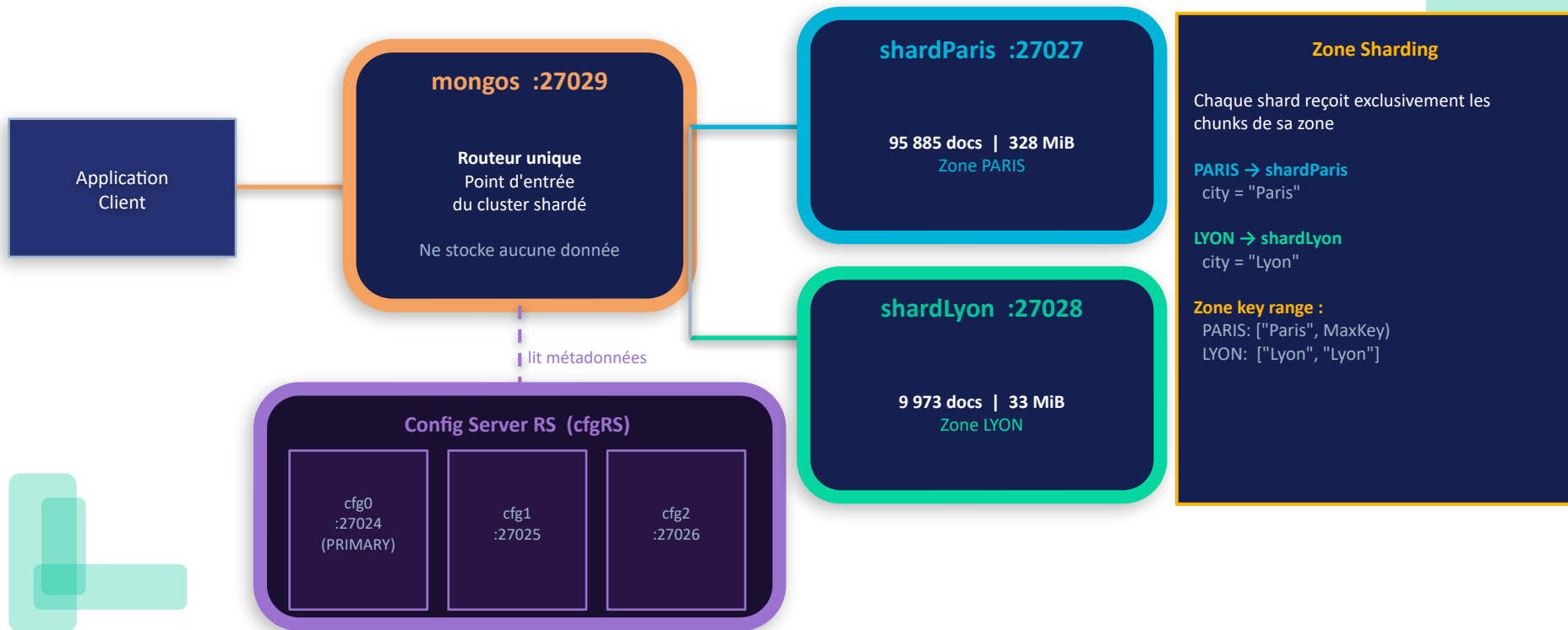


Balancer MongoDB : surveille le nombre de chunks par shard et déplace automatiquement des chunks pour équilibrer la charge. **Jumbo chunk** : chunk trop grand pour être divisé (ex. si tous les docs ont la même valeur de shard key → évité ici par le compound key).

3-4 Distribution : Sharding par zone géographique

Client → mongos (routeur) → cfgRS (métadonnées) + Shards (données)

Architecture Shardée — Vue d'ensemble



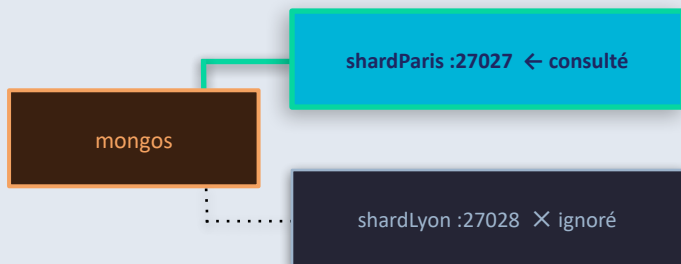
3-4 Distribution : Sharding par zone géographique

Utiliser la shard key dans les requêtes = performance optimale

✓ Requête CIBLÉE (Targeted Query)

```
db.listings.find({ city: "Paris" })
```

→ 1 shard seulement

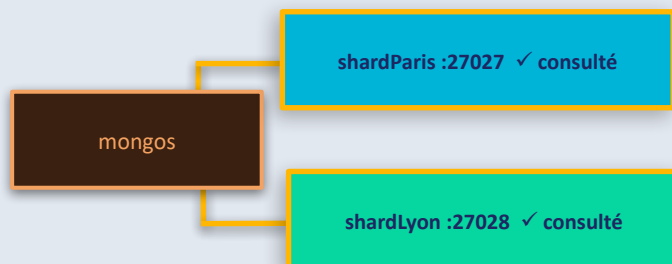


Performance optimale
1 seul shard consulté — résultat direct
Clé de shard présente dans le filtre WHERE

⚠ Requête SCATTER-GATHER (à éviter sur gros volume)

```
db.listings.find({ price: { $lt: 50 } })
```

→ TOUS les shards



Performance dégradée à grande échelle
Tous les shards consultés → résultats fusionnés par mongos
Acceptable sur petit volume — à éviter si partiel sur shard key

3-4 Distribution : Sharding par zone géographique

7 étapes pour passer d'un ReplicaSet à un cluster shardé par zone géographique

Mise en place du Sharding
— Séquence de démarrage

3-4 Distribution : Sharding par zone géographique

7 étapes pour passer d'un ReplicaSet à un cluster shardé par zone géographique

Mise en place du Sharding
— Séquence de démarrage

1

cfgRS
(Config Server)

3 nœuds config
élisent un PRIMARY
Stockent métadonnées

Commandes

```
mkdir rs{cfg0..2}  
mongod --configsvr  
--replSet cfgRS  
--port 27024..26  
rs.initiate(cfgRS)
```

Metadata
cfgRS actif

3-4 Distribution : Sharding par zone géographique

7 étapes pour passer d'un ReplicaSet à un cluster shardé par zone géographique

Mise en place du Sharding
— Séquence de démarrage

1

cfgRS
(Config Server)

3 nœuds config
élisent un PRIMAIRE
Stockent métadonnées

Démarrage

```
moylig@DESKTOP-CUIQHFG:/mnt/c/Users/moymo/OC/P7$ mkdir -p ~/mongodb/cfg0 ~/mongodb/cfg1 ~/mongodb/cfg2

moylig@DESKTOP-CUIQHFG:/mnt/c/Users/moymo/OC/P7$ mongod --configsvr --replSet cfgRS --port 27024 --dbpath ~/mongodb/cfg0 --fork --logpath ~/mongodb/cfg0/mongod.log --bind_ip_all
about to fork child process, waiting until server is ready for connections.
forked process: 1542
child process started successfully, parent exiting
moylig@DESKTOP-CUIQHFG:/mnt/c/Users/moymo/OC/P7$ mongod --configsvr --replSet cfgRS --port 27025 --dbpath ~/mongodb/cfg1 --fork --logpath ~/mongodb/cfg1/mongod.log --bind_ip_all
about to fork child process, waiting until server is ready for connections.
forked process: 1642
child process started successfully, parent exiting
moylig@DESKTOP-CUIQHFG:/mnt/c/Users/moymo/OC/P7$ mongod --configsvr --replSet cfgRS --port 27026 --dbpath ~/mongodb/cfg2 --fork --logpath ~/mongodb/cfg2/mongod.log --bind_ip_all
about to fork child process, waiting until server is ready for connections.
forked process: 1742
child process started successfully, parent exiting
```

Commandes

```
mkdir rs{cfg0..2}
mongod --configsvr
--replSet cfgRS
--port 27024..26
rs.initiate(cfgRS)
```

Initialisation

```
moylig@DESKTOP-CUIQHFG:/mnt/c/Users/moymo/OC/P7$ mongosh --port 27024 --eval '
rs.initiate({
  _id: "cfgRS",
  configsvr: true,
  members: [
    { _id: 0, host: "localhost:27024" },
    { _id: 1, host: "localhost:27025" },
    { _id: 2, host: "localhost:27026" }
  ]
})'
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1773740216, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1773740216, i: 1 })
}
```

Metadata
cfgRS actif

3-4 Distribution : Sharding par zone géographique

7 étapes pour passer d'un ReplicaSet à un cluster sharded par zone géographique

Mise en place du Sharding
— Séquence de démarrage

1

cfgRS
(Config Server)

3 nœuds config
élisent un PRIMARY
Stockent métadonnées

Commandes

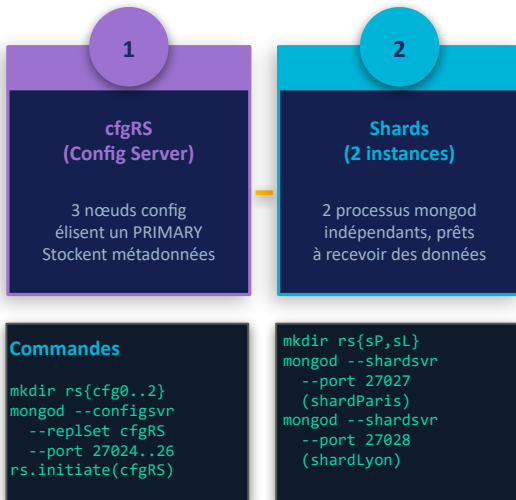
```
mkdir rs{cfg0..2}  
mongod --configsvr  
--replSet cfgRS  
--port 27024..26  
rs.initiate(cfgRS)
```

Metadata
cfgRS actif

3-4 Distribution : Sharding par zone géographique

7 étapes pour passer d'un ReplicaSet à un cluster shardé par zone géographique

Mise en place du Sharding
— Séquence de démarrage



Metadata
cfgRS actif

Shards
démarrés

3-4 Distribution : Sharding par zone géographique

7 étapes pour passer d'un ReplicaSet à un cluster shardé par zone géographique

Mise en place du Sharding
— Séquence de démarrage

1

cfgRS
(Config Server)

3 nœuds config
élisent un PRIMARY
Stockent métadonnées

2

Shards
(2 instances)

2 processus mongod

g --bind_ip_all
about to fork child process, waiting until server is ready for connections.
forked process: 2747

child process started successfully, parent exiting

moylig@DESKTOP-CUIQHFQ:/mnt/c/Users/moymo/OC/P7\$ mongod --shardsvr --replSet shardLyon --port 27028 --dbpath ~/mongodb/shard_lyon --fork --logpath ~/mongodb/shard_lyon/mongod.log --bind_ip_all

about to fork child process, waiting until server is ready for connections.

forked process: 2825

child process started successfully, parent exiting

--port 27028

(shardLyon)

Démarrage des shards

```
moylig@DESKTOP-CUIQHFQ:/mnt/c/Users/moymo/OC/P7$ mkdir -p ~/mongodb/shard_paris ~/mongodb/shard_lyon
```

Commandes

```
mkdir rs{cfg0..2}  
mongod --configsvr  
--replSet cfgRS  
--port 27024.26  
rs.initiate(cfgRS)
```

Initialisation des shards

Metadata
cfgRS actif

Shards
démarrés

```
moylig@DESKTOP-CUIQHFQ:/mnt/c/Users/moymo/OC/P7$ mongosh --port 27027 --eval 'rs.initiate({ _id: "shardParis", members: [{ _id: 0, host: "localhost:27027" }] })'
```

```
{  
  ok: 1,  
  '$clusterTime': {  
    clusterTime: Timestamp({ t: 1773740775, i: 1 }),  
    signature: {  
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),  
      keyId: Long('0')  
    }  
  },  
  operationTime: Timestamp({ t: 1773740775, i: 1 })  
}  
moylig@DESKTOP-CUIQHFQ:/mnt/c/Users/moymo/OC/P7$ mongosh --port 27028 --eval 'rs.initiate({ _id: "shardLyon", members: [{ _id: 0, host: "localhost:27028" }] })'
```

```
{  
  ok: 1,  
  '$clusterTime': {  
    clusterTime: Timestamp({ t: 1773740794, i: 1 }),  
    signature: {  
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),  
      keyId: Long('0')  
    }  
  },  
  operationTime: Timestamp({ t: 1773740794, i: 1 })  
}
```

3-4 Distribution : Sharding par zone géographique

7 étapes pour passer d'un ReplicaSet à un cluster shardé par zone géographique

Mise en place du Sharding
— Séquence de démarrage

1

cfgRS (Config Server)

3 nœuds config
élisent un PRIMARY
Stockent métadonnées

2

Shards (2 instances)

2 processus mongod
indépendants, prêts
à recevoir des données

Commandes

```
mkdir rs{cfg0..2}
mongod --configsvr
--replSet cfgRS
--port 27024..26
rs.initiate(cfgRS)
```

```
mkdir rs{sP,sL}
mongod --shardsvr
--port 27027
(shardParis)
mongod --shardsvr
--port 27028
(shardLyon)
```

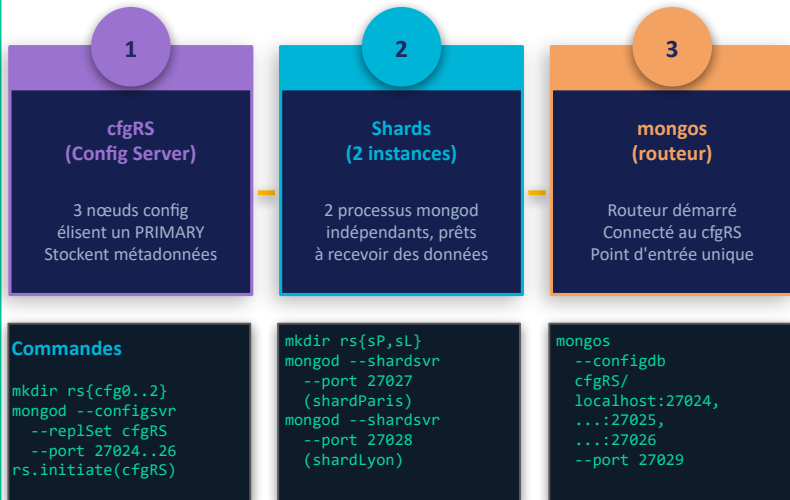
Metadata
cfgRS actif

Shards
démarrés

3-4 Distribution : Sharding par zone géographique

7 étapes pour passer d'un ReplicaSet à un cluster shardé par zone géographique

Mise en place du Sharding
— Séquence de démarrage



Metadata
cfgRS actif

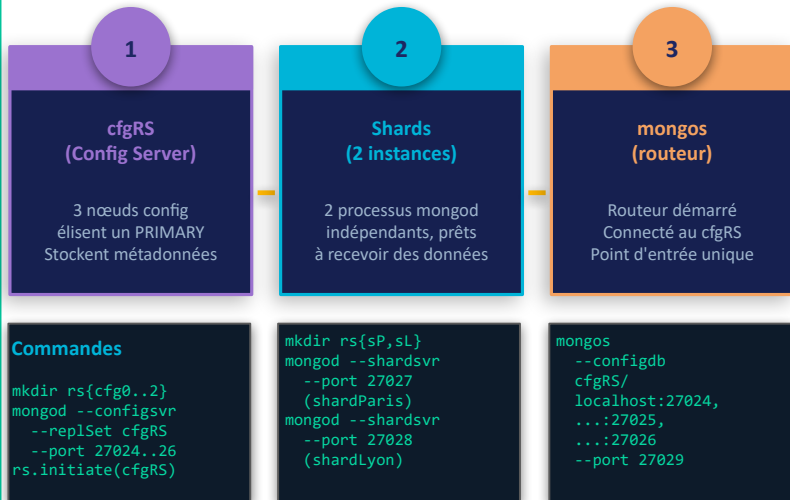
Shards
démarrés

Routeur
mongos actif

3-4 Distribution : Sharding par zone géographique

7 étapes pour passer d'un ReplicaSet à un cluster shardé par zone géographique

Mise en place du Sharding
— Séquence de démarrage



```
moylig@DESKTOP-CUIQHFQ:/mnt/c/Users/moymo/OC/P7$ mongos --configdb cfgRS/localhost:27024,localhost:27025,localhost:27026 --port 27029 --fork --logpath ~/mongodb/mongos.log --bind_ip_all
about to fork child process, waiting until server is ready for connections.
forked process: 3110
child process started successfully, parent exiting
```

Metadata
cfgRS actif

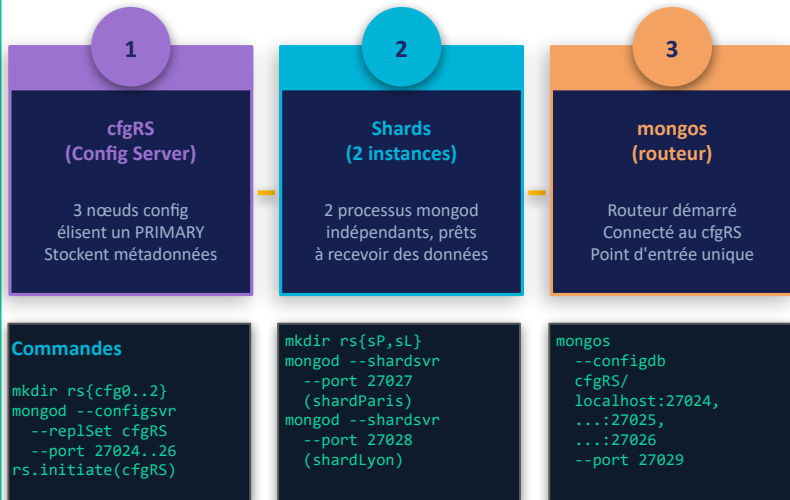
Shards
démarrés

Routeur
mongos actif

3-4 Distribution : Sharding par zone géographique

7 étapes pour passer d'un ReplicaSet à un cluster shardé par zone géographique

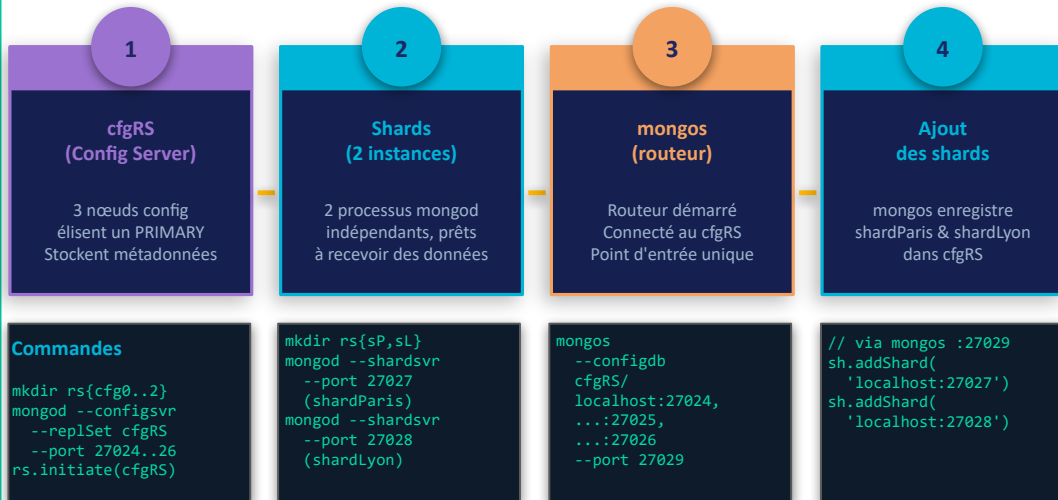
Mise en place du Sharding
— Séquence de démarrage



3-4 Distribution : Sharding par zone géographique

7 étapes pour passer d'un ReplicaSet à un cluster shardé par zone géographique

Mise en place du Sharding
— Séquence de démarrage



Metadata
cfgRS actif

Shards
démarrés

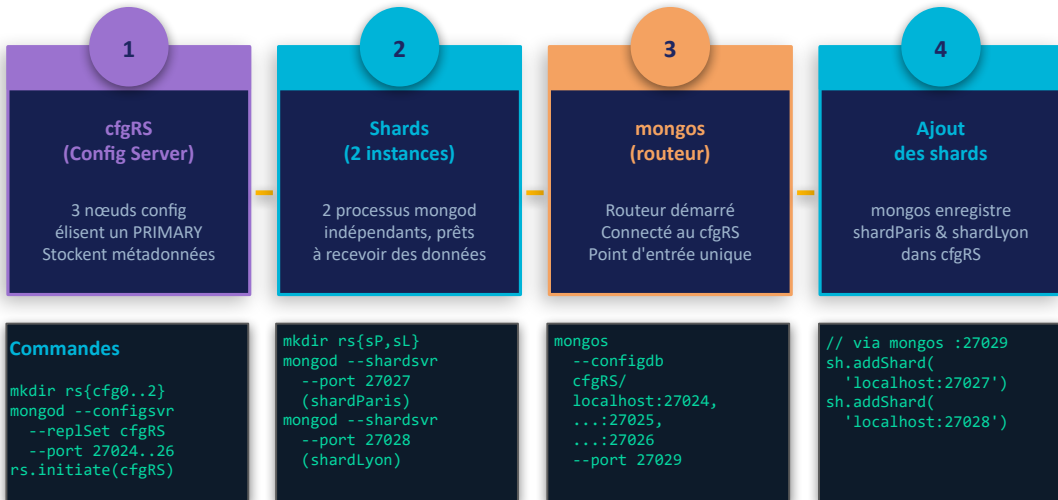
Routeur
mongos actif

Cluster
enregistré

3-4 Distribution : Sharding par zone géographique

7 étapes pour passer d'un ReplicaSet à un cluster sharded par zone géographique

Mise en place du Sharding
— Séquence de démarrage



Ajout des shards

```
[direct: mongos] test> sh.addShard("shardParis/localhost:27027")
{
  shardAdded: 'shardParis',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1773741185, i: 20 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1773741185, i: 20 })
}
[direct: mongos] test> sh.addShard("shardLyon/localhost:27028")
{
  shardAdded: 'shardLyon',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1773741193, i: 25 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1773741193, i: 19 })
}
```

Metadata
cfgRS actif

Shards
démarrés

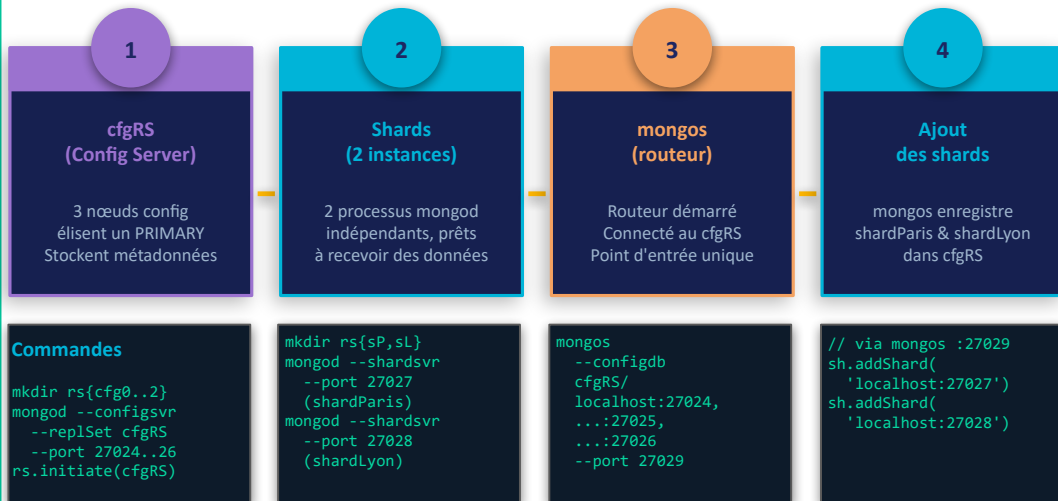
Routeur
mongos actif

Cluster
enregistré

3-4 Distribution : Sharding par zone géographique

7 étapes pour passer d'un ReplicaSet à un cluster shardé par zone géographique

Mise en place du Sharding
— Séquence de démarrage



Metadata
cfgRS actif

Shards
démarrés

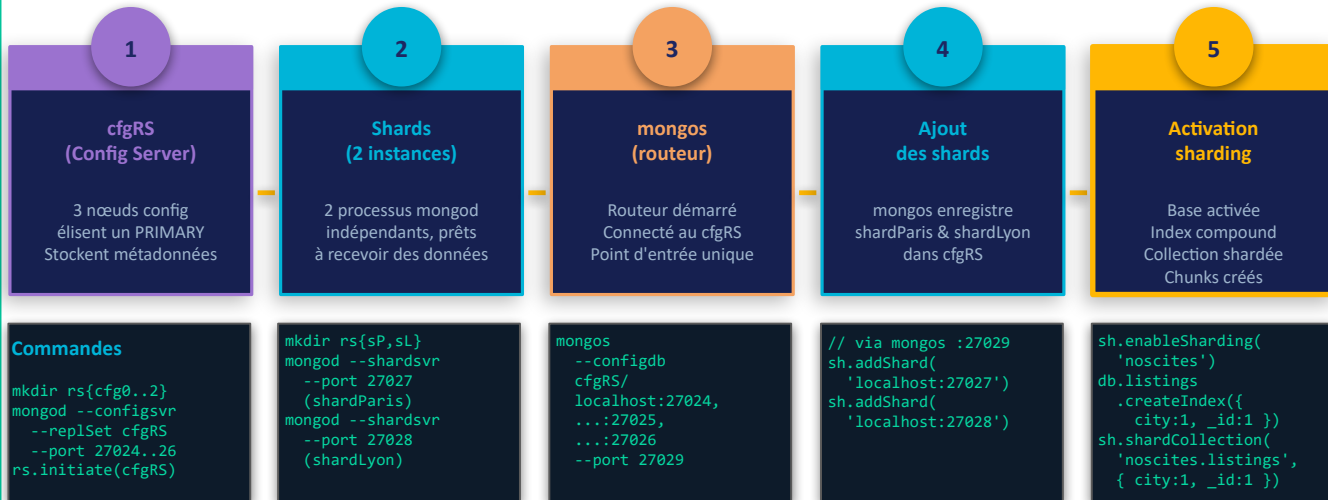
Routeur
mongos actif

Cluster
enregistré

3-4 Distribution : Sharding par zone géographique

7 étapes pour passer d'un ReplicaSet à un cluster shardé par zone géographique

Mise en place du Sharding
— Séquence de démarrage



Metadata
cfgRS actif

Shards
démarrés

Routeur
mongos actif

Cluster
enregistré

Collection
shardée

3-4 Distribution : Sharding par zone géographique

7 étapes pour passer d'un ReplicaSet à un cluster shardé par zone géographique

Mise en place du Sharding
— Séquence de démarrage

Activation du sharding

1

cfgRS (Config Server)

3 nœuds config
élisent un PRIMARY
Stockent métadonnées

Commandes

```
mkdir rs{cfg0..2}
mongod --configsvr
--port 27027
--replSet cfgRS
--port 27024..26
rs.initiate(cfgRS)
```

2

Shards (2 instances)

2 processus mongod
indépendants, prêts
à recevoir des données

```
mkdir rs{sP,sL}
mongod --shardsvr
--port 27027
(shardParis)
mongod --shardsvr
--port 27028
(shardLyon)
```

3

mongos (routeur)

Routeur démarré
Connecté au cfgRS
Point d'entrée unique

```
mongos
--configdb
cfgRS/
localhost:27024,
...:27025,
...:27026
--port 27029
```

4

Ajout des shards

mongos enregistre
shardParis & shardLyon
dans cfgRS

```
// via mongos :27029
sh.addShard(
'localhost:27027')
sh.addShard(
'localhost:27028')
```

5

Activation sharding

```
[direct: mongos] test> sh.enableSharding("noscites")
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1773741239, i: 9 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    },
    operationTime: Timestamp({ t: 1773741239, i: 6 })
  },
  db: 'noscites',
  sh: {
    { city:1, _id:1 }
  }
}
```

Création de l'index &
shardisation de la collection

```
[direct: mongos] test> use noscites
switched to db noscites
[direct: mongos] noscites> db.listings.createIndex({ city: 1, _id: 1 })
city_1_id_1
[direct: mongos] noscites> db.adminCommand({ shardCollection: "noscites.listings", key: { city: 1, _id: 1 } })
{
  collectionsSharded: 'noscites.listings',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1773741386, i: 29 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    },
    operationTime: Timestamp({ t: 1773741386, i: 29 })
  },
  db: 'noscites',
  sh: {
    { city:1, _id:1 }
  }
}
```

Metadata
cfgRS actif

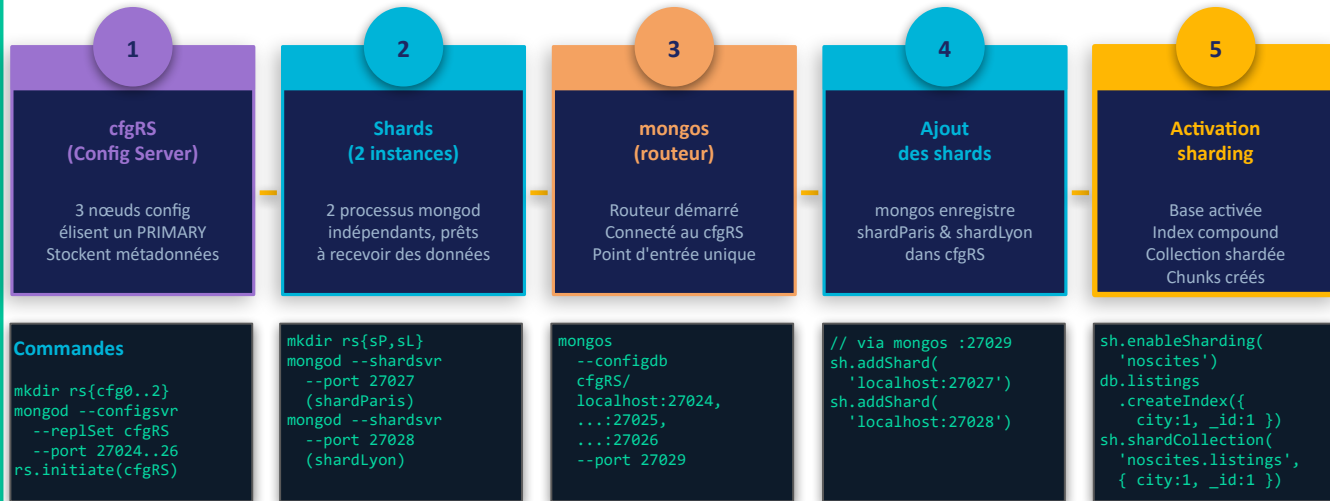
Shards
démarrés

Routeur
mongos actif

3-4 Distribution : Sharding par zone géographique

7 étapes pour passer d'un ReplicaSet à un cluster shardé par zone géographique

Mise en place du Sharding
— Séquence de démarrage



Metadata
cfgRS actif

Shards
démarrés

Routeur
mongos actif

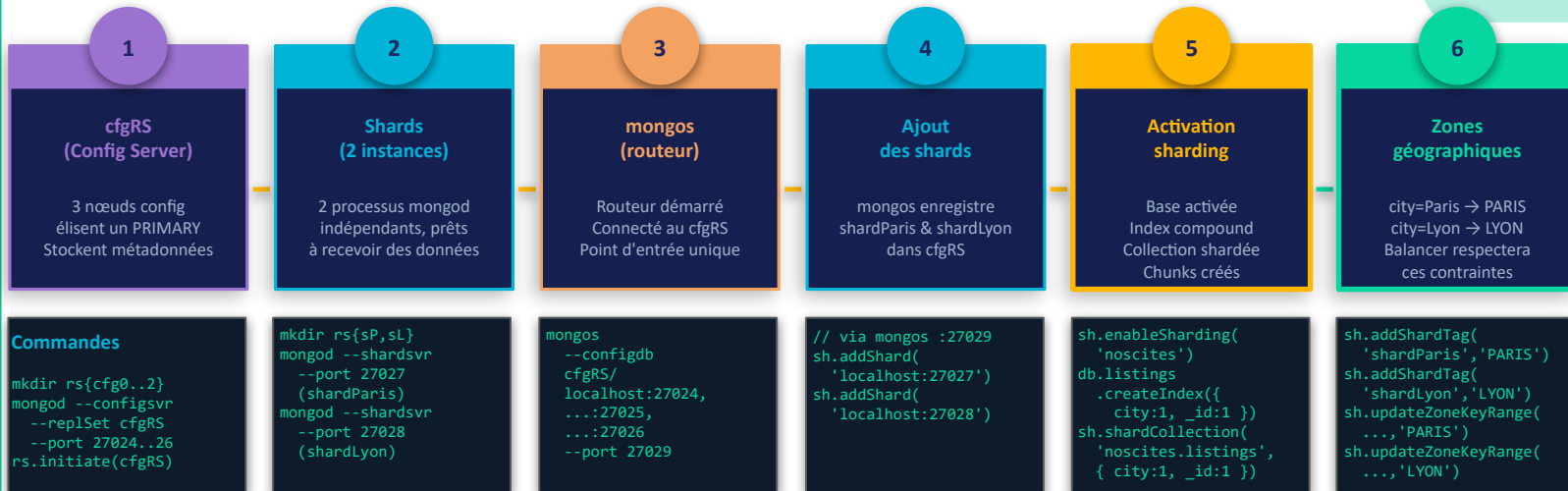
Cluster
enregistré

Collection
shardée

3-4 Distribution : Sharding par zone géographique

7 étapes pour passer d'un ReplicaSet à un cluster shardé par zone géographique

Mise en place du Sharding
— Séquence de démarrage



Metadata
cfgRS actif

Shards
démarrés

Routeur
mongos actif

Cluster
enregistré

Collection
shardée

Zones
configurées

3-4 Distribution : Sharding par zone géographique

7 étapes pour passer d'un ReplicaSet à un cluster sharded par zone géographique

Mise en place du Sharding
— Séquence de démarrage



Commandes

```
mkdir rs{cfg0..2}
mongod --configsvr
--port 27027
--replSet cfgRS
--port 27024..26
rs.initiate(cfgRS)
```

```
mkdir rs{sP,sL}
mongod --shardsvr
--port 27027
(shardParis)
mongod --shardsvr
--port 27028
(shardLyon)
```

```
mongos
--configdb
cfgRS/
localhost:27024,
...:27025,
...:27026
--port 27029
```

```
// via mongos :27029
sh.addShard(
'localhost:27027')
sh.addShard(
'localhost:27028')
```

```
[direct: mongos] noscites> sh.addShardToZone("shardParis", "PARIS")
sh.enal{
'nosce
db.list
.crea
cit
sh.shar
'nosce
{ cit
ok: 1,
'$clusterTime': {
clusterTime: Timestamp({ t: 1773741761, i: 1 }),
signature: {
hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
keyId: Long('0')
}
},
operationTime: Timestamp({ t: 1773741761, i: 1 })
}
[direct: mongos] noscites> sh.addShardToZone("shardLyon", "LYON")
ok: 1,
'$clusterTime': {
clusterTime: Timestamp({ t: 1773741770, i: 1 }),
signature: {
hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
keyId: Long('0')
}
},
operationTime: Timestamp({ t: 1773741770, i: 1 })
}
```

Définition des zones géographiques

Metadata
cfgRS actif

Shards
démarrés

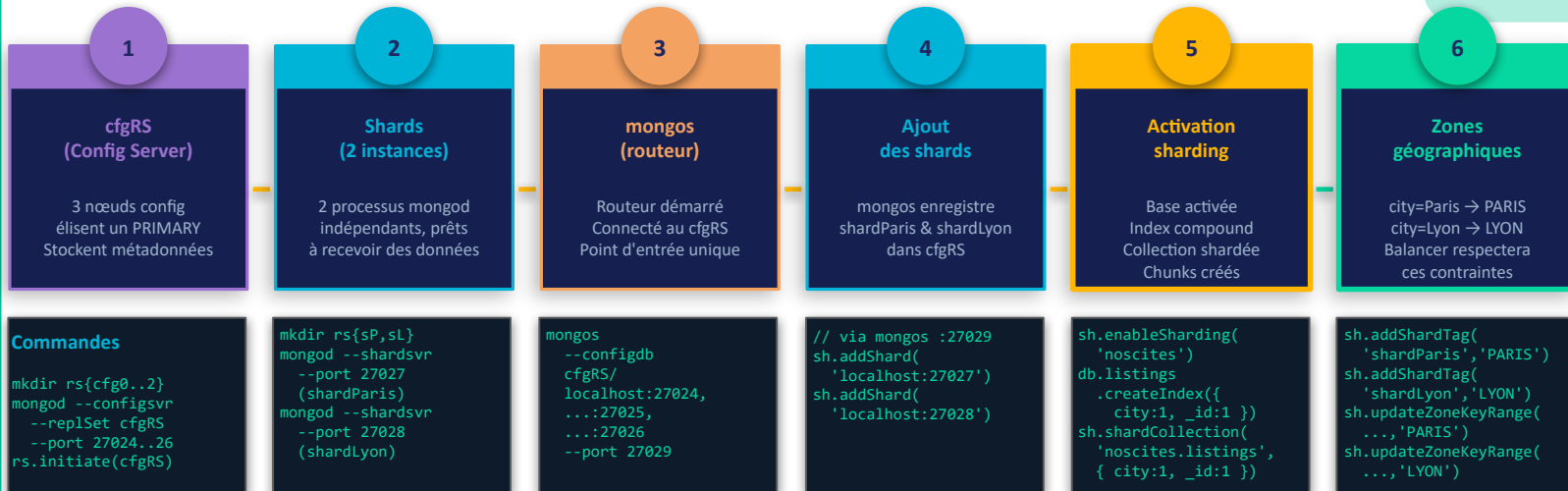
Routeur
mongos actif

Cluster
enregistré

3-4 Distribution : Sharding par zone géographique

7 étapes pour passer d'un ReplicaSet à un cluster shardé par zone géographique

Mise en place du Sharding
— Séquence de démarrage



Metadatas
cfgRS actif

Shards
démarrés

Routeur
mongos actif

Cluster
enregistré

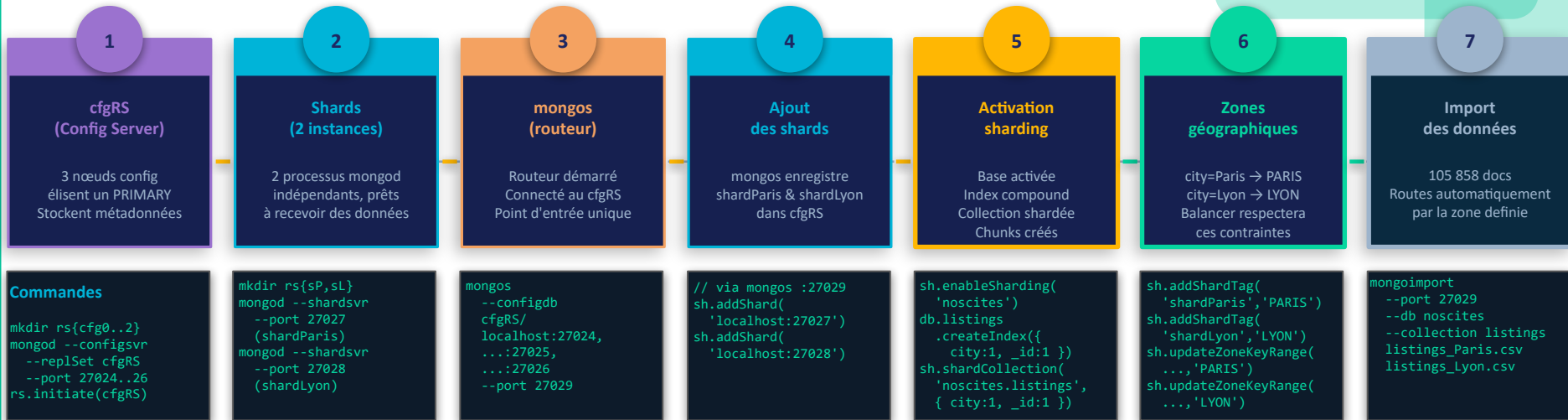
Collection
shardée

Zones
configurées

3-4 Distribution : Sharding par zone géographique

7 étapes pour passer d'un ReplicaSet à un cluster shardé par zone géographique

Mise en place du Sharding
— Séquence de démarrage



Metadata
cfgRS actif

Shards
démarrés

Routeur
mongos actif

Cluster
enregistré

Collection
shardée

Zones
configurées

Données
distribuées

3-4 Distribution : Sharding par zone géographique

7 étapes pour passer d'un ReplicaSet à un cluster sharded par zone géographique

Mise en place du Sharding
— Séquence de démarrage



Commandes

```
mkdir rs{cfg0..2}
mongod --configsvr
--port 27027
--replSet cfgRS
--port 27024..26
rs.initiate(cfgRS)
```

```
mkdir rs{sP,sL}
mongod --shardsvr
--port 27027
(shardParis)
mongod --shardsvr
--port 27028
(shardLyon)
```

```
mongos
--configdb
cfgRS/
localhost:27024,
...:27025,
...:27026
--port 27029
```

```
moylig@DESKTOP-CUIQHFQ:/mnt/c/Users/moymo/OC/P7$ mongoimport \
--port 27029 \
--db noscites \
--collection listings \
--type csv \
--headerline \
--file "/mnt/c/Users/moymo/OC/P7/listings_Paris_tagged.csv"
2026-03-17T11:23:01.266+0100 connected to: mongod://localhost:27029/
2026-03-17T11:23:04.266+0100 [####.....] noscites.listings 38.3MB/186MB (20.6%)
2026-03-17T11:23:07.266+0100 [#####.....] noscites.listings 76.4MB/186MB (41.2%)
2026-03-17T11:23:10.266+0100 [#####.....] noscites.listings 114MB/186MB (61.4%)
2026-03-17T11:23:13.266+0100 [#####.....] noscites.listings 152MB/186MB (81.9%)
2026-03-17T11:23:16.021+0100 [#####.....] noscites.listings 186MB/186MB (100.0%)
2026-03-17T11:23:16.022+0100 95885 document(s) imported successfully. 0 document(s) failed to import.
```

Import des données dans le cluster shardé

Metadata
cfgRS actif

Shard
démarré

```
moylig@DESKTOP-CUIQHFQ:/mnt/c/Users/moymo/OC/P7$ mongoimport \
--port 27029 \
--db noscites \
--collection listings \
--type csv \
--headerline \
--file "/mnt/c/Users/moymo/OC/P7/listings_Lyon_tagged.csv"
2026-03-17T11:24:07.820+0100 connected to: mongod://localhost:27029/
2026-03-17T11:24:09.278+0100 9973 document(s) imported successfully. 0 document(s) failed to import.
```

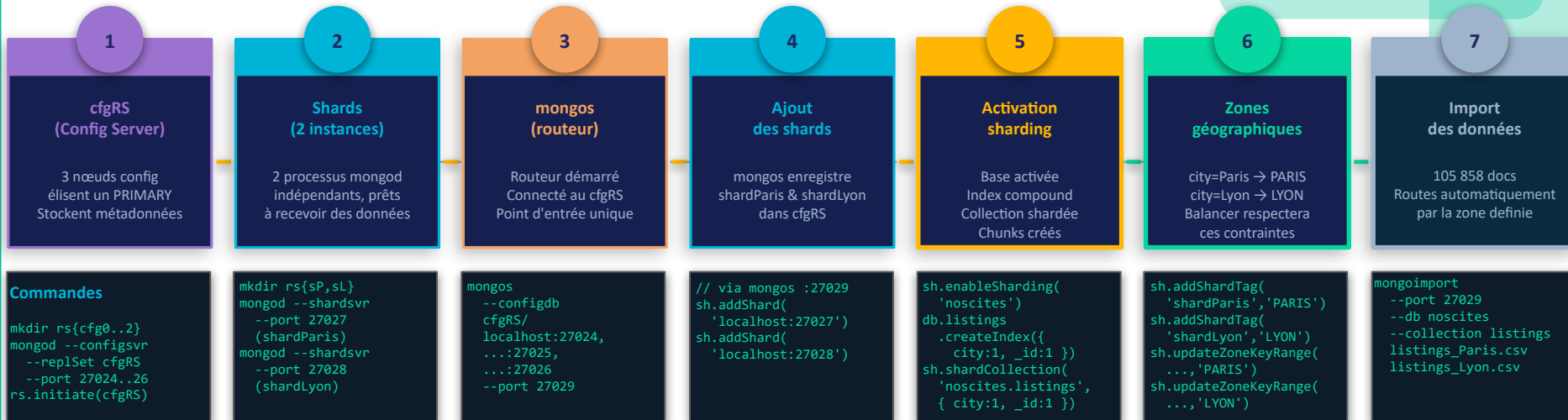
Zones
configurées

Données
distribuées

3-4 Distribution : Sharding par zone géographique

7 étapes pour passer d'un ReplicaSet à un cluster shardé par zone géographique

Mise en place du Sharding
— Séquence de démarrage



Metadatas
cfgRS actif

Shards
démarrés

Routeur
mongos actif

Cluster
enregistré

Collection
shardée

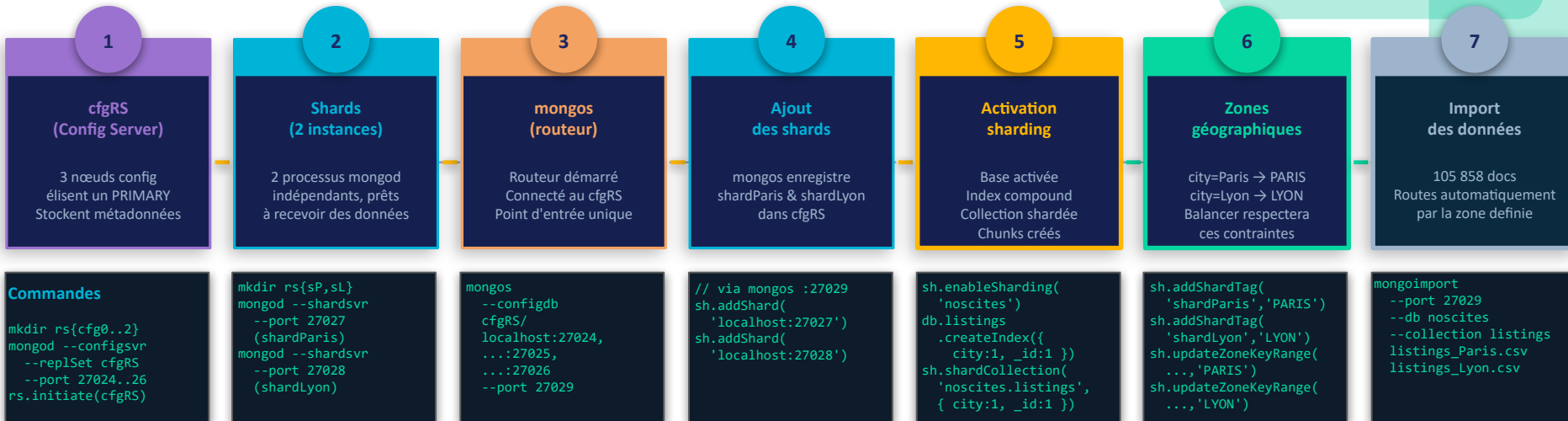
Zones
configurées

Données
distribuées

3-4 Distribution : Sharding par zone géographique

7 étapes pour passer d'un ReplicaSet à un cluster shardé par zone géographique

Mise en place du Sharding
— Séquence de démarrage



Metadata
cfgRS actif

Shards
démarrés

Routeur
mongos actif

Cluster
enregistré

Collection
shardée

Zones
configurées

Données
distribuées

Ordre critique : cfgRS doit être initialisé **avant** mongos (qui en a besoin au démarrage). **Shards + mongos** avant **sh.addShard()**. **Import** toujours via mongos :27029 (jamais directement sur un shard).

3-4 Distribution : Sharding par zone géographique

Ce qui se passe dans chaque composant lors de la mise en place du cluster

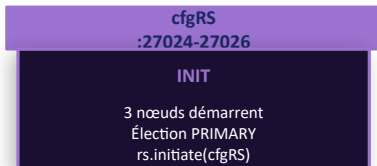
**Sharding — Flux d'initialisation
& rôles des composants**

3-4 Distribution : Sharding par zone géographique

Ce qui se passe dans chaque composant lors de la mise en place du cluster

Sharding — Flux d'initialisation
& rôles des composants

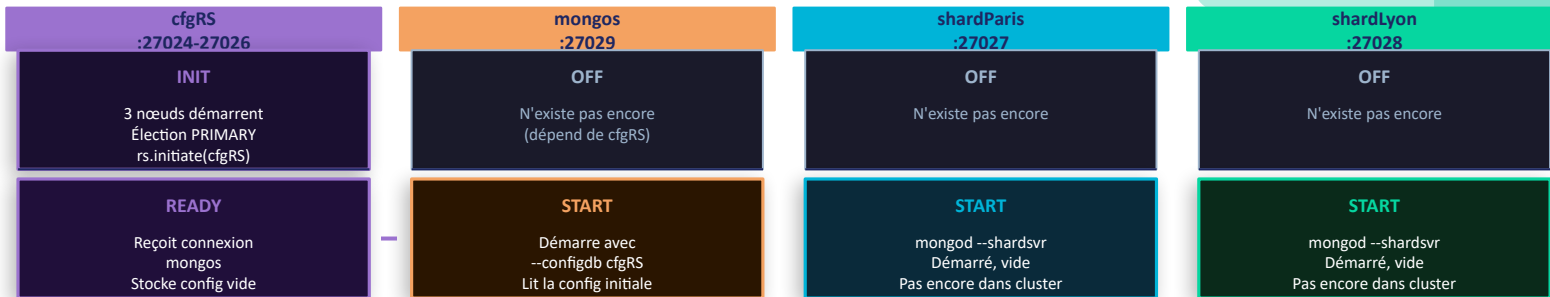
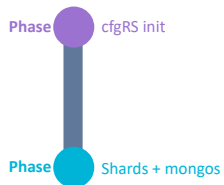
Phase  cfgRS init



3-4 Distribution : Sharding par zone géographique

Ce qui se passe dans chaque composant lors de la mise en place du cluster

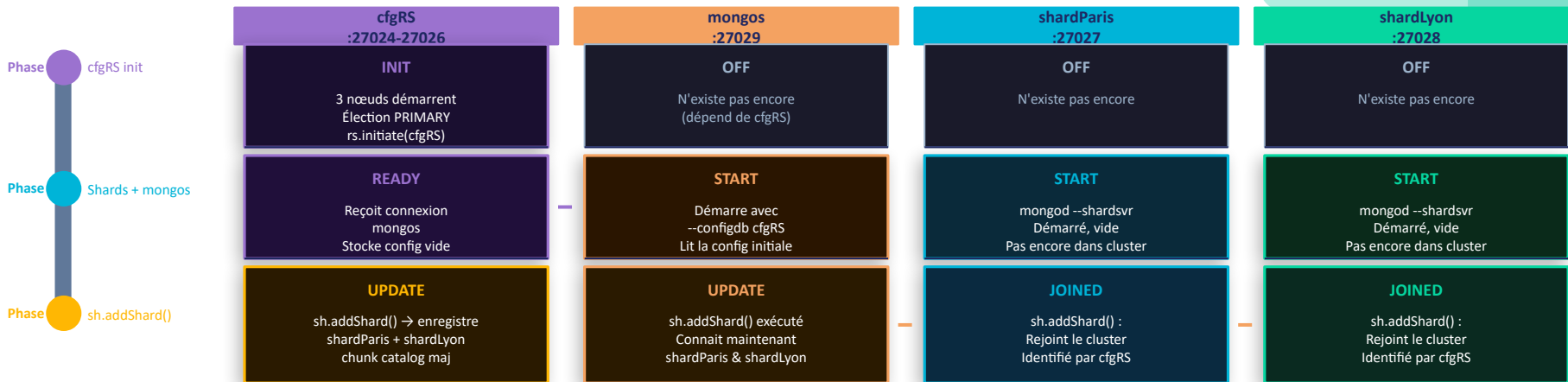
Sharding — Flux d'initialisation & rôles des composants



3-4 Distribution : Sharding par zone géographique

Ce qui se passe dans chaque composant lors de la mise en place du cluster

Sharding — Flux d'initialisation & rôles des composants



3-4 Distribution : Sharding par zone géographique

Ce qui se passe dans chaque composant lors de la mise en place du cluster

Sharding — Flux d'initialisation & rôles des composants



3-4 Distribution : Sharding par zone géographique

Ce qui se passe dans chaque composant lors de la mise en place du cluster

Sharding — Flux d'initialisation & rôles des composants



3-4 Distribution : Sharding par zone géographique

Ce qui se passe dans chaque composant lors de la mise en place du cluster

Sharding — Flux d'initialisation & rôles des composants



Clé de lecture : chaque colonne montre l'état interne du composant à chaque phase. **cfgRS** est la source de vérité du cluster. **mongos** ne fait que router — aucune donnée stockée. **shards** n'acceptent de données qu'après sh.addShard() + zones. **Les 11**

3-4 Distribution : Sharding par zone géographique

Vérification finale

État global du cluster

```
[direct: mongos] noscites> sh.status()
shardingVersion
{ _id: 1, clusterId: ObjectId('69b920c5c1b729f423dc1258') }
---
shards
[
  {
    _id: 'shardLyon',
    host: 'shardLyon/localhost:27028',
    state: 1,
    topologyTime: Timestamp({ t: 1773741193, i: 10 }),
    replSetConfigVersion: Long('-1'),
    tags: [ 'LYON' ]
  },
  {
    _id: 'shardParis',
    host: 'shardParis/localhost:27027',
    state: 1,
    topologyTime: Timestamp({ t: 1773741185, i: 10 }),
    replSetConfigVersion: Long('-1'),
    tags: [ 'PARIS' ]
  }
]
---
active mongoses
[ { '8.0.19': 1 } ]
---
```


3-4 Distribution : Sharding par zone géographique

Vérification finale

```
[direct: mongos] noscites> db.listings.getShardDistribution()
Shard shardLyon at shardLyon/localhost:27028
{
  data: '33.33MiB',
  docs: 9973,
  chunks: 4,
  'estimated data per chunk': '8.33MiB',
  'estimated docs per chunk': 2493
}
---
Shard shardParis at shardParis/localhost:27027
{
  data: '328.49MiB',
  docs: 95885,
  chunks: 1,
  'estimated data per chunk': '328.49MiB',
  'estimated docs per chunk': 95885
}
---
Totals
{
  data: '361.82MiB',
  docs: 105858,
  chunks: 5,
  'Shard shardLyon': [
    '9.21 % data',
    '9.42 % docs in cluster',
    '3KiB avg obj size on shard'
  ],
  'Shard shardParis': [
    '90.78 % data',
    '90.57 % docs in cluster',
    '3KiB avg obj size on shard'
  ]
}
```

3-4 Distribution : Sharding par zone géographique

Vérification finale

```
moylig@DESKTOP-CUIQHFG:/mnt/c/Users/moymo/OC/P7$ mongosh --port 27029 --eval '
use("noscites");
print("=== Routage automatique via mongos :27029 ===");
print("shardParis → Paris :", db.listings.countDocuments({city:"Paris"}));
print("shardLyon → Lyon :", db.listings.countDocuments({city:"Lyon"}));
print("Total      (mongos):", db.listings.countDocuments());
'

=== Routage automatique via mongos :27029 ===
shardParis → Paris : 95885
shardLyon → Lyon : 9973
Total      (mongos): 105858
```

✓ **Sharding opérationnel** : Chaque ville est isolée sur son shard dédié. Le routeur mongos (port 27027) dirige automatiquement les requêtes vers le bon shard selon la valeur du champ city. countDocuments() via mongos retourne les valeurs correctes : Paris 95885, Lyon 9973, Total 105858.

3-4 Distribution : Sharding par zone géographique

Vérification finale

Composant	Port	Zone	Rôle et résultat
Config Server RS (cfgRS)	27024–27026	—	3 nœuds RS (évite SPOF). Métadonnées des chunks et zones
Shard Paris	27027	PARIS	✅ 95 885 docs Paris (90,78%) 328,49 MiB Zone city="Paris"
Shard Lyon	27028	LYON	✅ 9 973 docs Lyon (9,21%) 33,33 MiB Zone city="Lyon"
mongos (routeur)	27029	—	Point d'entrée unique. Route city="Paris" → shardParis city="Lyon" → shardLyon
Clé de shard	—	—	{city:1, _id:1} — composée (évite les jumbo chunks)

4- Synthèse

Cartographie finale

Composant	Port(s)
noscitesRS PRIMARY	27017
noscitesRS SEC 1	27018
noscitesRS SEC 2	27019
noscitesRS ARBITRE	27020
cfgRS nœud 1	27024
cfgRS nœud 2	27025
cfgRS nœud 3	27026
shardParis	27027
shardLyon	27028
mongos	27029

4- Synthèse

Résultats clés à retenir

Indicateur	Paris	Lyon
Annonces totales	95 885	9 973
Hôtes uniques	71 979	7 703
Super hôtes	10 027 (13.9%)	1 331 (17.3%)
Résa instantanée	22 094 (23.0%)	2 112 (21.2%)
Type dominant	Entire home/apt (89.4%)	Entire home/apt
Quartier le + dense	Buttes-Montmartre (10 555)	—
Quartier taux résa le + haut	Ménilmontant (75.4%)	—
Médiane avis	3	—
Médiane avis super hôtes	24	—

5- Utilisation pratique

Script de démarrage

```
moylig@DESKTOP-CUIQHFG:/mnt/c/Users/moymo/OC/P7$ bash start_mongodb.sh
```

MongoDB NosCités P7 – Démarrage

[1/2] ReplicaSet noscitesRS (ports 27017-27020)

```
about to fork child process, waiting until server is ready for connections.
forked process: 1298
child process started successfully, parent exiting
about to fork child process, waiting until server is ready for connections.
forked process: 1395
child process started successfully, parent exiting
about to fork child process, waiting until server is ready for connections.
forked process: 1495
child process started successfully, parent exiting
about to fork child process, waiting until server is ready for connections.
forked process: 1599
child process started successfully, parent exiting
i Attente du PRIMARY noscitesRS (15s max)...
✓ ReplicaSet noscitesRS opérationnel
✓ PRIMARY → localhost:27017
✓ SECONDARY1 → localhost:27018
✓ SECONDARY2 → localhost:27019
✓ ARBITRE → localhost:27020
```

Démarrage terminé – Récapitulatif

```
Connexion ReplicaSet : mongosh --port 27017
Connexion Sharding : mongosh --port 27029
```

Commandes utiles :

```
Vérifier RS : mongosh --port 27017 --eval 'rs.status()'
Vérifier Shard : mongosh --port 27029 --eval 'sh.status()'
Distribution : mongosh --port 27029 --eval 'use("noscites"); db.listings.getShardDistribution()'
```

```
Pour arrêter : bash stop_mongodb.sh
```

[2/2] Cluster Shardé

```
→ Config Server RS cfgRS (ports 27024-27026)...
about to fork child process, waiting until server is ready for connections.
forked process: 1719
child process started successfully, parent exiting
about to fork child process, waiting until server is ready for connections.
forked process: 1834
child process started successfully, parent exiting
about to fork child process, waiting until server is ready for connections.
forked process: 1961
child process started successfully, parent exiting
i Attente élection PRIMARY cfgRS (15s max)...
✓ cfgRS PRIMARY élu sur port 27024
→ Shards (ports 27027-27028)...
about to fork child process, waiting until server is ready for connections.
forked process: 2112
child process started successfully, parent exiting
about to fork child process, waiting until server is ready for connections.
forked process: 2404
child process started successfully, parent exiting
→ Routeur mongos (port 27029)...
about to fork child process, waiting until server is ready for connections.
forked process: 2590
child process started successfully, parent exiting
i Attente du routeur mongos (10s max)...
✓ Cluster shardé opérationnel
✓ cfgRS → localhost:27024
✓ shardParis → localhost:27027
✓ shardLyon → localhost:27028
✓ mongos → localhost:27029
```

5- Utilisation pratique

Vérification de status

```
moylig@DESKTOP-CUIQHFG:/mnt/c/Users/moymo/OC/P7$ bash status_mongodb.sh
```

```
MongoDB NosCités P7 – Statut des services

— ReplicaSet noscitesRS —
✓ UP rs0 (port 27017) – PRIMARY (priorité 2)
✓ UP rs1 (port 27018) – SECONDARY 1
✓ UP rs2 (port 27019) – SECONDARY 2
✓ UP rs3 (port 27020) – ARBITRE (vote seul)

— Config Server RS (cfgRS) —
✓ UP cfg0 (port 27024) – cfgRS nœud 0 – PRIMARY
✓ UP cfg1 (port 27025) – cfgRS nœud 1
✓ UP cfg2 (port 27026) – cfgRS nœud 2

— Shards —
✓ UP shardParis (port 27027) – zone PARIS – 95 885 docs
✓ UP shardLyon (port 27028) – zone LYON – 9 973 docs

— Routeur —
✓ UP mongos (port 27029) – Point d'entrée sharding

— Résumé —
Services actifs : 10 / 10

Processus système : 9 mongod + 1 mongos
```

5- Utilisation pratique

Script d'arrêt

```
moylig@DESKTOP-CUIQHFG:/mnt/c/Users/moymo/OC/P7$ bash stop_mongodb.sh
```

MongoDB NosCités P7 – Arrêt

⚠ L'ordre d'arrêt est important pour éviter les erreurs

[1/3] Arrêt mongos (port 27029)

✓ mongos (port 27029) arrêté

[2/3] Arrêt des shards

✓ shardParis (port 27027) arrêté

✓ shardLyon (port 27028) arrêté

[3/3] Arrêt du Config Server RS (cfgRS)

✓ cfgRS nœud 0 (port 27024) arrêté

✓ cfgRS nœud 1 (port 27025) arrêté

✓ cfgRS nœud 2 (port 27026) arrêté

[RS] Arrêt du ReplicaSet noscitesRS

✓ SECONDARY 2 (port 27019) arrêté

✓ SECONDARY 1 (port 27018) arrêté

✓ ARBITRE (port 27020) arrêté

✓ PRIMARY (port 27017) arrêté

Arrêt terminé – Vérification

✓ Aucun processus MongoDB actif – arrêt complet

Pour redémarrer : bash start_mongodb.sh

Conclusion

"que se passe-t-il si un serveur tombe ?" ⇒ **ReplicaSet**

"que se passe-t-il si les données ne tiennent plus sur un seul serveur ?" ⇒ **Sharding**

Conclusion

Le ReplicaSet garantit la **robustesse** du système : redondance des données et continuité de service en cas de panne.

Le sharding assure la **scalabilité** : la capacité à distribuer un volume croissant de données sur plusieurs serveurs, avec un routage transparent via mongos.

Conclusion

Point à explorer.

Combiner les deux : chaque shard est lui-même un ReplicaSet pour cumuler robustesse et scalabilité.